Logbook

January 5, 2026

Review of HPC Resource Management and Research Presentation

Today, I explored recent research directions in High-Performance Computing by reviewing a call for papers on Dynamic Resource Management. To better understand the topic and communicate my findings, I created a slide deck for presentation to the research group. The slides focused on the key problems and challenges identified in recent work, particularly from the paper *Network-Centered Resource Management for HPC Networks* .

One of the main issues highlighted in both the paper and the slides was the severe underutilization of modern supercomputers. Despite their high theoretical performance, actual usage remains very low, mainly due to communication bottlenecks in the network. The interconnection network has become a critical limiting factor, especially with communication-heavy workloads such as large-scale AI training .

A major problem identified is that current resource management approaches are not designed with the network as a primary consideration. Most existing systems rely on static assumptions such as fixed job sizes, predefined communication patterns, and static routing strategies. However, in practice, workloads and system conditions change over time, making these assumptions unrealistic. This mismatch leads to inefficient allocation of resources and poor overall system performance .

Another key issue is the lack of flexibility in current HPC systems. Jobs are typically static, meaning that once resources are allocated at the start, they remain fixed throughout execution. The paper pointed out that supporting elastic or malleable jobs—where resource usage can change at runtime—could significantly improve system utilization. However, this is difficult to implement due to limitations in the existing software stack, particularly communication libraries such as MPI .

The research also emphasized the challenge of accurately modeling network behavior. As shown in the discussion of HPC networks, modern systems use complex topologies and adaptive routing, which increase the difficulty of building models that are both accurate and computationally efficient. Traditional analytical models are often too simplistic, while detailed simulations are too slow for real-time use. This creates a gap where better modeling approaches are needed for effective decision-making .

Another important problem is the scale of the resource allocation search space. Even with static allocation, the number of possible configurations is extremely large. When dynamic or elastic resource allocation is introduced, the problem becomes even more complex. This makes it necessary to design new algorithms that can efficiently explore the search space and make decisions within practical time limits .

The paper also pointed out that current resource managers rarely use real-time network information when making decisions. Instead, they rely on static or pre-profiled data. As a result, they cannot respond effectively to changing conditions such as congestion, workload variation, or hardware issues. This highlights the need for resource managers that incorporate live monitoring and adapt their decisions accordingly .

Finally, the inability to adapt to dynamic system behavior was identified as a critical limitation. Since both workloads and system conditions evolve over time, resource allocation decisions made at the start of execution may become suboptimal. Addressing this requires a shift toward dynamic, closed-loop systems that continuously monitor the system and adjust resource allocation during runtime .

Overall, preparing the slide deck helped me clearly understand the main research problems in dynamic resource management for HPC systems. It also allowed me to present these ideas in a structured way to the research group, focusing on the limitations of current approaches and the need for more adaptive, network-aware solutions.

January 8, 2026

Research Group Discussion and Feedback

Today, I presented the slide deck on dynamic resource management in HPC to the research group. The discussion was useful in

understanding how feasible the idea would be in practice. The general feedback was that the direction is strong and relevant, but implementing a full system would require significant resources, especially in terms of infrastructure and experimentation.

Based on this, it became clear that I need to narrow down the scope and identify a more focused problem within this area. Instead of attempting a broad system-level solution, the goal should be to find a specific niche that is both manageable and meaningful from a research perspective.

During the discussion, Ian suggested looking into the paper *Dynamic Adapting Scheduling for HPC: Eliminating Job Failure through Robust Resource Allocation*. This work focuses on job failures in HPC systems, which introduces a more concrete and constrained problem within the broader context of resource management.

Following this suggestion, I decided to review a set of papers related to job failure and robust scheduling in HPC. The aim is to understand how resource allocation strategies can be designed to improve system reliability, and whether this can serve as a suitable direction for my work.

January 13, 2026

Review of Job Scheduling and Fault Tolerance Papers

Today, I went through the paper Ian suggested, along with several related works on HPC job scheduling. Most of the papers I looked at focused on fault-tolerant scheduling, and I noticed that many of them were quite heavy on mathematical modeling and formal proofs. This made them more challenging to follow, especially in terms of understanding how the ideas translate into practical implementations.

A key theme across these papers was handling failures in HPC systems through scheduling strategies. Some of the work approached this by modeling the HPC cluster network and predicting potential failures. Based on these predictions, the network could become asymmetric due to failed components, which would affect communication and performance.

What I found particularly interesting was that some approaches tried to address this by applying algorithms to restore symmetry in the

network, even in the presence of failures. This involves adjusting the routing or allocation strategy while considering the failed components. However, this also introduces another layer of complexity, as it requires selecting and understanding an additional algorithm specifically for handling network asymmetry.

At this point, I need to better understand how these different approaches fit together and whether this direction is practical for my work. I plan to discuss these observations with Ian and Prof. Ivanov to get their input on whether this is a suitable path to explore further.

January 15, 2026

Refining Research Direction Based on Group Feedback

Today, I presented my recent findings on HPC job scheduling and fault-tolerant approaches to the group. During the discussion, it became clear that my current direction was moving away from the group’s primary focus on reliability.

Ian pointed out that my approach was leaning more toward computer science aspects, particularly algorithms and scheduling, rather than focusing on the underlying hardware. He suggested that I shift my perspective toward electrical engineering concerns, especially the reliability and aging of hardware components. This includes looking at how components degrade over time and how that impacts system performance.

Following this, Prof. Andre suggested that I explore Network-on-Chip (NoC) systems, particularly in the context of aging. This was helpful because it still connects to my earlier interest in network behavior and system performance, but grounds it more in hardware-level reliability.

Based on this discussion, I decided to start looking into NoC architectures and study how aging affects their performance and reliability. This seems like a better-aligned direction with the group’s focus while still allowing me to build on some of the concepts I have already explored.

January 20, 2026

Thesis Direction and Initial Research Framing

Today, I decided to focus my thesis on adaptive routing algorithms for Network-on-Chip (NoC) systems using reinforcement learning. This gave the thesis a clearer direction and helped narrow down the broader area of NoC routing into a more specific research problem that appears both relevant and technically promising.

I began by reviewing the general landscape of adaptive routing methods in NoC architectures. In particular, I looked at papers published through IEEE and studied the paper *A Survey of Adaptive Routing Techniques in Two-Dimensional Network-on-Chip Architectures*. This helped me build a broader understanding of how adaptive routing has evolved, what kinds of design goals are usually considered, and where the current limitations lie. From this reading, it became clear that routing decisions in many adaptive approaches are still strongly dependent on predefined rules or heuristic methods.

I then paid closer attention to recent aging-aware routing algorithms. These works showed that aging is an important reliability concern in NoC systems, especially as long-term operation affects interconnects and routers. I noted that many of these algorithms make routing decisions based on heuristics that try to balance traffic while also reducing the stress on components that are more vulnerable to wear-out over time. While these heuristic approaches are useful, they also seem to have limitations because they rely on manually designed decision rules, which may not adapt well to changing network conditions or different aging patterns.

At the same time, I also examined fault-tolerant routing work in which reinforcement learning had already been applied. These papers were encouraging because they demonstrated that reinforcement learning can be used to improve routing quality and, in some cases, achieve lower latency. This suggested that learning-based decision making could be a strong alternative to fixed heuristic routing, especially in situations where the network condition changes over time or where multiple competing objectives need to be balanced.

Based on these observations, I started shaping the central idea of my thesis: combining reinforcement learning with aging-aware routing for 2D NoC architectures. The main motivation behind this idea was that reinforcement learning may be able to learn routing policies that respond not only to current traffic conditions but also to long-term reliability concerns caused by hardware aging. Instead of relying only on handcrafted rules, the routing strategy could potentially adapt its behavior based on the condition of the network and the expected impact of aging on different paths.

I also identified the aging models that I wanted to consider in this work: Electromigration (EM), Hot Carrier Injection (HCI), and Bias Temperature Instability (BTI). These models are widely discussed in reliability studies (BTI not so much) and together provide a stronger basis for representing aging effects in NoC components. Including these three models should make the research more realistic and technically grounded, since each one captures a different degradation mechanism that can influence long-term routing performance and network lifetime.

Another important decision I made today was that the work should be built on top of state-of-the-art aging-aware routing algorithms rather than starting from a completely isolated baseline. This would allow the thesis to remain connected to current research and make the contribution easier to position. My goal is not only to propose a reinforcement learning-based method, but also to show how it compares against established aging-aware routing strategies and whether the learning-based approach can offer measurable improvements in latency, adaptability, and long-term reliability.

By the end of the day, I had a much clearer problem statement in mind. The thesis direction now centered on investigating whether reinforcement learning can be integrated into aging-aware adaptive routing for 2D NoC systems, using EM, HCI, and BTI as the underlying aging considerations. This felt like a strong and meaningful research direction because it connects two active areas of study: intelligent routing and hardware reliability. It also appeared to offer enough depth for both theoretical study and practical evaluation.

January 21, 2026

AROMa: Aging-Aware Deadlock-Free Adaptive Routing Algorithm and Online Monitoring in 3D NoCs

1. Is the paper related to routing in NoC?

Yes. It is directly about routing in NoC, more specifically **adaptive routing in 3D NoCs**. The paper proposes **AROMa**, an **aging-aware, deadlock-free adaptive routing algorithm** that changes routing paths over time based on the aging condition of routers. Routing is the core contribution of the work, although it is tightly coupled with an online aging monitoring system.

2. What problem is being solved?

The paper tries to solve the problem of **uneven aging in 3D NoC routers**. Because some routers are used more often and because temperature differs across layers in a 3D stack, routers do not age uniformly. This leads to **performance degradation, higher delay, and eventually timing failure** in the most stressed routers. The authors want to reduce the age of the most aged routers and balance aging across the network, while still using shortest paths and keeping the routing deadlock-free.

3. Why is the problem interesting in the first place?

The problem matters because NoCs are central to many-core systems, and 3D NoCs are attractive for improving area, performance, power, and scalability. But these benefits are threatened by reliability issues, especially **aging caused by BTI and HCI**. Aging increases transistor threshold voltage, slows circuits down, and can eventually cause timing failure. In 3D NoCs this is even more interesting because temperature is not uniform across layers, so the same traffic pattern can produce different aging behavior in different parts of the chip. That makes aging a serious practical concern, not just a theoretical one.

4. Why is the problem challenging to solve?

The problem is hard for several reasons.

First, aging is a **runtime phenomenon**. It depends on both **temperature** and **stress**, and those depend on the actual traffic flowing through routers over time. So an offline method cannot fully capture what happens during long execution.

Second, in 3D NoCs, layers have different thermal profiles. The paper shows that routers in different layers can experience different temperatures even with similar stress, so the same amount of traffic does not imply the same aging everywhere. This makes the problem more complex than in 2D NoCs.

Third, changing routes to avoid aged routers must not break the network. The routing must remain **minimal enough for performance** and also **deadlock-free**. That means the solution has to balance reliability improvement with correctness and low overhead.

5. What key idea(s) are being exploited to solve this problem?

The paper uses two main ideas.

a. Online aging monitoring with D-CAT

The authors propose **Distributed Centralized Aging Tables (D-CATs)**, one per layer. These tables estimate router aging using two traffic-derived quantities:

- **number of flits**
- **residence time of flits inside the router**

The idea is that flits are the only real stimuli inside a router, so by tracking flit count and residence time, the system can estimate stress, temperature, and then aging rate. Since different layers have different temperature behavior, each layer gets its own D-CAT. This lets the system track aging online without depending directly on the running application type.

b. Periodically rerouting traffic through less aged shortest paths

AROMa computes **k-best shortest paths** between each source and destination pair, then periodically chooses paths that **avoid the maximum-aged routers** and prefer paths with lower total router age. This gives heavily aged routers time to recover, especially from BTI-related degradation. In simple terms, the network keeps using good short paths, but it rotates traffic away from routers that are aging too quickly.

c. Deadlock-free adaptive routing

Because arbitrary path switching can create deadlocks, the paper also proves that the routing remains deadlock-free by using **static virtual-channel allocation**. For 3D NoC, the proof uses **four virtual channels per port** and partitions shortest-path flows into subsets that conform to deadlock-free turn models.

6. How is the solution evaluated?

The solution is evaluated mainly through **simulation**. The authors implement AROMa in **gem5 full-system simulation mode** and run **PARSEC and SPLASH-2 benchmarks**. They compare AROMa against:

- **NAW**: non-aging-aware routing, using XY in 2D and XYZ in 3D
- **OFAR**: an offline aging-aware routing approach based on prior work

They evaluate both **2D and 3D NoCs**, specifically a **4×8 2D mesh** and a **4×4×2 3D mesh**, over **three years of simulated execution time.** They also use **McPAT** for power estimation and **HotSpot** for thermal modeling.

7. What metrics are used for evaluation?

The main evaluation metrics are:

a. Maximum aged router delay degradation

This is the age of the most aged router in the network. The goal is to reduce this maximum because it is the router most likely to fail first.

b. Age imbalance

This is the difference between the maximum-aged and minimum-aged routers. It shows how evenly the network ages. Lower imbalance means better load distribution from an aging perspective.

c. Energy-Delay-Product-Per-Flit (EDDPF)

This is used as an overhead metric to study the efficiency cost of the routing approach.

d. Network latency

They compare latency under uniform random traffic and observe saturation behavior.

e. Link utilization

This is used to study how traffic load is distributed across the network.

f. Area and memory overhead

They measure added hardware cost for the monitoring logic and storage cost for D-CAT.

8. What are the limitations of this work?

A few limitations stand out.

a. It estimates temperature indirectly

The paper says router temperature is estimated using flit count and residence time, but it **does not consider the temperature impact of surrounding circuits**. The authors themselves mention this as future work. So the thermal model is useful, but simplified.

b. It focuses on router aging, not the whole system equally

The method mainly targets **router and link aging**. It does not appear to model all system-level reliability effects in the same depth, such as broader interactions with processing elements or memory structures.

c. It depends on periodic table updates and precomputed D-CATs

The approach needs D-CAT construction using McPAT and HotSpot, plus periodic monitoring and routing-table updates. Even though the reported overhead is small, this still adds architectural complexity.

d. Deadlock freedom requires multiple VCs

Their proof for 3D NoC assumes **four virtual channels per port**. That may be acceptable, but it is still a design requirement and may not fit every router design. They also note that other work reports a lower minimum VC count for full adaptivity, though with tradeoffs.

e. The evaluation is simulation-based

The work is strong on simulation, but it is still simulation. There is no silicon prototype or FPGA-style experimental validation here.

9. What ideas for future work arise from this paper?

Several good future directions come from this paper.

a. Better thermal modeling

The clearest one is the one the authors mention: include the effect of **surrounding circuits' temperatures** instead of estimating temperature only from local flit count and residence time.

b. Extend to other aging mechanisms

This paper focuses on **BTI and HCI**. A natural next step would be to include other aging and wear mechanisms, especially if the goal is a more complete reliability model.

c. Apply the idea to 2D and 3D NoCs with more realistic system variation

The approach could be extended to systems with different traffic behaviors, heterogeneous routers, irregular topologies, or fault-prone environments where aging and faults are considered together. The paper already touches related work in fault tolerance, so combining both directions seems promising.

d. Smarter path selection methods

The routing policy picks among k-best shortest paths using age information. A future direction could be more advanced decision-making that jointly optimizes aging, latency, energy, and congestion instead of mainly prioritizing aging-aware shortest-path selection.

e. Hardware implementation study

Since the paper claims negligible area overhead, it would be useful to validate the method in a more concrete hardware setting and study timing, synthesis results, and practical integration issues.

Overall takeaway

Yes, this is clearly a **NoC routing paper**. The core problem is **how to route traffic in a way that reduces and balances aging in 3D NoC routers**, where aging depends on traffic stress and layer temperature. The main contribution is the combination of **online aging monitoring through D-CAT** and **adaptive shortest-path routing that avoids over-aged routers while remaining deadlock-free**. The paper is strong because it ties together reliability modeling, routing design, and evaluation in a fairly complete way. Its biggest weakness is that the monitoring and temperature model are still simplified compared to real full-chip behavior.

January 22, 2026

ARTEMIS: An Aging-Aware Runtime Application Mapping Framework for 3D NoC-Based Chip Multiprocessors

1. Is the paper related to routing in NoC?

Yes, but routing is **not the main focus** of the paper.

The paper is mainly about **runtime application mapping and voltage scaling** for **3D NoC-based CMPs** under aging and power constraints. However, it does include a routing component: the authors propose a **symmetric aging-enabled routing path allocation (SAR)** heuristic for the NoC, with the goal of balancing aging between NoC routers and compute cores. So the paper is related to routing in NoC, but routing is one part of a larger system-level framework.

2. What problem is being solved?

The paper solves the problem of **how to extend the useful lifetime of a 3D NoC-based chip multiprocessor at runtime**, while still meeting:

- application performance constraints,
- chip-wide dark-silicon power budget constraints,
- and reliability constraints caused by aging.

More specifically, the authors argue that lifetime management should not look only at **BTI-induced circuit aging** in cores and routers. It should also consider **electromigration (EM)-induced aging in the power delivery network (PDN)**, because increasing supply voltage to compensate for BTI can worsen PDN aging. Their ARTEMIS framework tries to jointly manage:

- **core aging**,
- **NoC router aging**,
- **PDN aging**,
- **application mapping**,
- and **runtime DVS decisions**.

3. Why is the problem interesting in the first place?

The problem is interesting because aging directly affects whether future manycore chips, especially **3D CMPs**, can remain reliable and useful over time.

The paper highlights two linked issues:

- **BTI** slows down transistors and degrades the performance of cores and NoC routers.
- **EM** increases resistance in PDN wires, causing larger IR drops and further slowing circuits.

In 3D chips this becomes more serious because 3D integration leads to:

- higher current densities,
- more thermal stress,
- and stricter power delivery challenges.

At the same time, designers also have to deal with **dark silicon**, meaning not all cores can be active at once because of power constraints. So the problem is not just “how to reduce aging,” but rather “how to keep the chip useful for longer while still running as many applications as possible under realistic power and performance limits.” That makes it practically important.

4. Why is the problem challenging to solve?

The problem is challenging because the different aging and performance effects are tightly coupled.

A few specific difficulties stand out:

- **BTI aging, PDN aging, voltage, and temperature interact with each other.** Raising voltage can help meet timing, but it increases power, raises current density, and accelerates PDN aging.
- **3D chips are power-limited.** The framework must obey a dark-silicon power budget, so it cannot simply keep increasing voltage or keep too many resources active.
- **Aging changes over time.** Mapping decisions that look good now may be poor later as the chip degrades.
- **Application requirements vary.** Different applications have different thread counts, communication patterns, and frequency constraints, so the system has to make runtime decisions instead of relying only on fixed offline choices.
- **The NoC must also be considered.** If only cores are considered, routers may age too quickly and make an otherwise useful region unusable.

So the challenge is really a joint optimization problem across reliability, performance, communication, voltage scaling, and power delivery.

5. What key idea(s) are being exploited to solve this problem?

The paper uses several connected ideas.

a. Joint management of circuit aging and PDN aging

This is the main idea of the paper. Instead of only managing BTI aging in cores, ARTEMIS also tracks the effect of **EM aging in the PDN**. The mapping heuristic prefers regions that can still meet frequency constraints **without heavily stressing already degraded power-delivery paths**.

b. Runtime application mapping onto contiguous 3D regions

Applications are mapped onto rectangular or cuboidal regions of the 3D CMP. The framework uses a region-selection heuristic that tries to choose regions with:

- enough performance headroom,
- lower worst-case IR drop,
- and a more favorable aging condition,
  while also respecting the dark-silicon power budget.

c. Voltage selection together with mapping

The framework does not treat voltage scaling separately. It jointly decides **where** to map and **which Vdd** to use, since both choices affect aging, power, and performance.

d. Symmetric Aging-Enabled Routing (SAR)

For the NoC, the authors propose SAR, which tries to balance aging between routers and their nearby cores. The routing cost function considers:

- router-core aging imbalance,
- and communication volume on links.

It uses this to pick routes that help the NoC age more evenly, while still keeping congestion under control.

e. Epoch-based runtime control with sensor feedback

The framework assumes periodic feedback from on-chip aging and voltage sensors. At the end of each epoch, it updates the chip’s aging profile and uses that updated state for future mapping and DVS decisions.

6. How is the solution evaluated?

The solution is evaluated through **simulation-based experiments**.

The authors use:

- **SPLASH-2** and **PARSEC** benchmarks,
- a **60-core 3D NoC-based CMP** with a **5×4×3** 3D mesh,
- application sequences categorized as **communication-intensive**, **compute-intensive**, and **mixed** workloads.

They profile application execution time, power, and parallelism using:

- **SNIPER** for multicore simulation,
- **McPAT** for power modeling,
- **3D-ICE** for thermal analysis,
- and **lp_solve** for PDN analysis.

They compare three ARTEMIS variants against two baselines:

- **WC-GB**: worst-case guard-banding with high fixed voltage,
- **WL+DVS**: wear-leveling plus dynamic voltage scaling,
- and ARTEMIS with different routing options including **XYZ**, **ACR**, and **SAR**.

7. What metrics are used for evaluation?

The main evaluation metrics are:

- **Total number of applications serviced over chip lifetime** — their main lifetime productivity metric.
- **Total CMP lifetime** in years.
- **Application throughput over lifetime** in applications per hour.
- **Average power dissipation per application**.
- **Maximum percent worst-case IR drop** at end of lifetime.
- **Variance of worst-case IR drop distribution** across the chip.
- **Mean effective VT degradation in compute cores** at end of lifetime.

They also use visual plots of:

- **VT degradation across the 3D chip**,
- and **worst-case IR drops across layers**,
  to show how evenly or unevenly the chip ages under different frameworks. The surface plots on pages 12–13 are especially useful for this comparison.

8. What are the limitations of this work?

A few limitations are fairly clear from the paper.

a. Routing is secondary

Although the paper includes SAR, the main problem is application mapping and DVS. So if your main interest is NoC routing itself, this paper is not as focused on routing design as a dedicated routing paper would be.

b. It depends on several modeling assumptions

The framework assumes:

- aging sensors and voltage sensors are available,
- execution times and communication volumes are known from offline profiling,
- applications map to contiguous cuboidal regions,
- thread migration is not considered,
- and one global chip-wide supply voltage is used.

These assumptions help make the framework manageable, but they simplify real systems.

c. Evaluation is simulation-based

The paper does not present hardware implementation or silicon validation. The claims are based on a detailed simulation framework, not measured chip data.

d. Some real-world variability is left out

The authors explicitly mention future work on **process variation** and **service queue waiting time**, which suggests these are not yet modeled in the current framework.

e. Limited routing diversity

SAR builds on a partially adaptive routing model with limited overhead, which is practical, but it also means the space of routing options is constrained compared to more aggressive adaptive methods.

9. What ideas for future work arise from this paper?

The paper itself suggests a couple of direct future directions:

- **Include process variation** in the framework.
- **Consider service-queue waiting time** of applications, not just how many applications are ultimately serviced.

Beyond that, some natural follow-up ideas are:

- **More realistic runtime models** with uncertain workloads instead of relying so much on offline profiling.
- **Finer-grained voltage control**, such as per-region or per-core voltage domains, if overheads can be controlled.
- **Joint routing and mapping optimization** with stronger focus on NoC behavior, especially under communication-heavy workloads.
- **Inclusion of more aging mechanisms**, not just BTI in circuits and EM in PDN.
- **Validation on hardware prototypes** or with lower-level implementation studies to test practical feasibility.

Overall takeaway

This paper is related to NoC routing, but it is better described as a **system-level aging-aware runtime management paper for 3D NoC-based CMPs**. Its main contribution is not a standalone routing algorithm, but a framework that jointly handles:

- application mapping,
- voltage scaling,
- core aging,
- router aging,
- and PDN aging.

The routing contribution, SAR, is important, but it supports the larger objective of increasing the useful lifetime of the whole chip.

January 25, 2026

Clotho: Proactive Wearout Deceleration in Chip-Multiprocessor Interconnects

1. Is the paper related to routing algorithm in NoC?

Yes. This paper is directly related to a **routing algorithm in NoC**. Its main contribution is **Clotho**, a **wearout-aware adaptive routing**

algorithm for NoC-based chip multiprocessors. The algorithm routes traffic away from routers and links that are more vulnerable to wearout.

2. What problem is being solved?

The paper addresses the problem of **how to prolong the lifetime of a NoC by reducing wearout before failures happen**. The focus is on two aging mechanisms that matter for the interconnect:

- **Electromigration (EM)** in links
- **Hot-Carrier Injection (HCI)** in routers

The authors argue that the NoC is especially fragile because even a single worn-out link or router can make the whole CMP unusable by causing disconnection, segmentation, or inaccessible peripherals. So the problem is to design a routing algorithm that **proactively reduces accumulated stress on vulnerable NoC components** instead of waiting for faults and routing around them afterward.

3. Why is the problem interesting in the first place?

The problem is interesting because **NoC failure is more critical than core failure** in many CMP systems. The paper makes this point early: a core can often fail and be bypassed or replaced logically, but a router or link failure can break communication paths and damage the usability of the whole chip. The examples in Figure 1 on page 1 show this clearly: a failed link can trigger deadlock, partition the network, or disconnect I/O even if most of the chip is still healthy.

It is also interesting because wearout is getting worse with technology scaling. The paper cites projections of a major increase in wear rate and argues that circuit-level solutions alone often cost too much in power or timing margin. That creates room for an architectural solution like routing.

4. Why is the problem challenging to solve?

The problem is difficult for several reasons.

First, **you cannot reduce total network activity without hurting usefulness**. Packets must still be delivered, so the goal is not to eliminate stress but to **redistribute it intelligently** away from the weakest parts of the network.

Second, wearout depends on **multiple interacting factors**:

- activity or utilization,
- temperature,
- and for HCI, process variation as well.

So the routing algorithm must reason about more than just traffic load. It has to consider which components are already vulnerable because of physical conditions, not only current usage.

Third, this information is not always local. A router may need to know the wear sensitivity of **distant paths**, which makes implementation harder. The paper says that unlike congestion, accumulated wear and temperature state must be explicitly communicated if a router is to make globally good decisions.

Finally, the algorithm must still be practical in hardware and avoid large runtime cost. That is why the authors also compare a global and a local version of the idea.

5. What key idea(s) are being exploited to solve this problem?

The core idea is to use **adaptive routing to avoid components that are more susceptible to wearout**, based on a physically informed estimate of lifetime.

More specifically, the paper exploits these ideas:

a. Use routing to redistribute long-term stress

Instead of reacting after faults happen, Clotho tries to **prevent early failures** by steering traffic away from links and routers that are more likely to wear out.

b. Model wear using EM and HCI lifetime equations

The routing decision is driven by lifetime models:

- EM is tied mainly to **link switching activity** and temperature.
- HCI is tied mainly to **router critical-path switching activity**, temperature, and process variation.

These models are turned into relative lifetime metrics through acceleration factors and path weights.

c. Combine activity, temperature, and process variation

This is an important point. The algorithm does not use traffic load alone. It combines:

- measured utilization,
- dynamic temperature,
- and process variation maps

to estimate which paths are safer to use.

d. Global wear-aware path selection

The main version, **Clotho-GAR**, uses a global view of the network. Routers share wear-related status information by piggybacking it in packet headers. The source router then chooses among minimal paths using a weighted lifetime metric that penalizes the weaker paths more strongly.

e. Compare global and local awareness

The paper also proposes **Clotho-LAR**, a local version that only looks at nearest-neighbor information. This is mainly useful as a lower-overhead reference point to show whether global awareness is worth it.

6. How is the solution evaluated?

The solution is evaluated through **full-system simulation**.

The authors use:

- **gem5** full-system simulator,
- **Ruby** shared-memory model,
- **Garnet** network simulator,
- and workloads from the **PARSEC** benchmark suite.

The target platform is a:

- **64-core CMP**
- with an **8×8 mesh NoC**.

They compare:

- **Clotho-GAR**,

- **Clotho-LAR**,
- against a baseline **deterministic dimension-order routing (DOR)** scheme.

Each simulation is repeated fifteen times with different temperature mappings, and the reported results are averaged.

7. What metrics are used for evaluation?

The main evaluation metrics are:

a. Acceleration factor for EM lifetime

This is the normalized **MTTF improvement for EM** compared to DOR. It shows how much longer the network lasts under Clotho relative to the baseline. Figure 4 reports this.

b. Acceleration factor for HCI lifetime under process variation

This is the normalized **MTTF improvement for HCI**, again relative to DOR. Figure 5 reports this.

c. Performance impact / speedup

The paper also measures the **performance cost** of the routing algorithm compared to DOR, based on total execution cycles. Figure 7 reports this.

d. Sensitivity to different EM/HCI weights

The authors also do a small **sensitivity analysis** by changing the relative weighting of EM and HCI in the combined routing cost. Figure 6 shows the effect on EM and HCI acceleration factors.

8. What are the limitations of this work?

A few limitations stand out.

a. It only targets EM and HCI

The paper explicitly says **NBTI is not the focus**. That is a clear scope choice, but it means the method does not address all important aging mechanisms.

b. Router temperature is simplified

The paper assumes a per-tile random temperature distribution mainly driven by the cores and states that the NoC contributes relatively little to total chip power. So the thermal model is simplified and does not deeply model router self-heating.

c. The local version is weak

Clotho-LAR gives very limited benefit and can even reduce lifetime under HCI on average. That suggests local wear-awareness is not enough and that the approach depends heavily on global knowledge.

d. There is some performance loss

Clotho-GAR improves lifetime, but it comes with about **2 percent average performance degradation**. That is small, but it is still a tradeoff.

e. It is still model- and simulation-driven

Like many architecture papers, this work relies on aging models and full-system simulation. There is no silicon or FPGA validation. Also, some constants in the physical models are proprietary, so the authors use relative lifetime comparison instead of absolute lifetime prediction.

f. Focuses on minimal-path routing

The implementation relies on progressive minimal paths and adapted shortest-path logic. That keeps things practical, but it may limit how much lifetime optimization is possible compared to a broader path search.

9. What ideas for future work arise from this paper?

Some natural future directions come directly from the gaps in the paper.

a. Include more aging mechanisms

Since this work excludes NBTI, a next step would be to build a routing method that handles **EM, HCI, and NBTI together**.

b. Better temperature modeling

A more detailed thermal model that includes stronger NoC thermal feedback or more realistic runtime thermal evolution could improve

the quality of the routing decisions.

c. Joint routing and fault tolerance

Clotho is proactive, while much earlier work is reactive. A useful extension would be to combine both:

- proactively slow wearout,
- then react gracefully once faults eventually appear.

That would make the full lifetime management story more complete.

d. Stronger hardware implementation study

The paper estimates hardware overhead and describes implementation ideas, but a fuller hardware validation would be valuable.

e. Explore broader routing policies

The paper compares global and local awareness, but future work could study:

- hybrid schemes,
- other adaptive routing strategies,
- or machine-learning-based policies using the same wear metrics.

f. Extend beyond mesh CMPs

Since the paper is built around an 8×8 mesh CMP, it would be interesting to test whether the same idea works as well for other NoC topologies or 3D NoCs.

Overall takeaway

Yes, this paper is clearly about a **NoC routing algorithm**. The main problem is **how to slow down wearout in NoC routers and links before failures occur**, especially under **EM and HCI**. The key idea is to use **adaptive routing guided by physical wear models, temperature, utilization, and process variation**, so that packets avoid the most vulnerable regions of the network. The strongest result is that **Clotho-GAR** improves lifetime substantially for EM and modestly for HCI, with a small performance cost.

January 27, 2026

Fault-Tolerant Routing Algorithm for Mesh based NoC using Reinforcement Learning

1. Is the paper related to routing algorithm in NoC?

Yes. This paper is directly about a **routing algorithm in NoC**. More specifically, it proposes a **fault-tolerant routing algorithm for mesh-based NoC** using **reinforcement learning**, particularly **Q-learning**.

2. What problem is being solved?

The paper tries to solve the problem of **routing packets correctly in a mesh NoC when links and routers can fail**. Since NoC components are vulnerable to faults, packets may not be able to follow normal shortest paths. The authors propose a routing method that can still find an **optimal or near-optimal path** from source to destination in the presence of:

- link faults,
- router faults,
- and combinations of both.

3. Why is the problem interesting in the first place?

The problem matters because NoCs are a core communication structure in many-core and MPSoC systems, and their scalability is one of the main reasons they replaced bus-based interconnects. But with higher switching activity and growing system size, NoC routers and links can fail due to heat and stress. If routing is not fault tolerant, packet delivery will break down and overall system performance will degrade. So the problem is important because reliability of communication directly affects whether a large multicore system remains usable.

4. Why is the problem challenging to solve?

The problem is challenging for a few reasons.

First, faults can occur at **different parts of the communication infrastructure**, not just one place. The paper considers both **router faults and link faults**, which makes routing decisions harder.

Second, many existing methods either:

- handle only limited fault scenarios,

- require added hardware or knowledge bases,
- or increase processing and area overhead.

Third, once faults appear, the routing algorithm still has to find a valid path without excessively increasing path length. In some cases, the best path is no longer the usual XY or YX path, so a more flexible method is needed.

5. What key idea(s) are being exploited to solve this problem?

The main idea is to model routing in mesh NoC as a **reinforcement learning problem**, specifically a **Markov Decision Process**, and then use **Q-learning** to learn the best routing actions.

The important ideas are:

a. Routers are modeled as states

Each router in the mesh is treated as a state. Moving from one router to a neighboring router is treated as a state transition.

b. Routing choices are modeled as actions

At each router, the possible outgoing directions are the available actions. Because routers in a mesh have different numbers of neighbors, the action space is not the same at every state.

c. Reward-guided learning

The reward table is designed so that allowed actions get rewards and the destination has the highest reward. By repeatedly updating Q-values, the agent learns which transitions are better for reaching the destination.

d. Q-learning for optimal path finding

After training over a number of epochs, the algorithm chooses the action with the highest Q-value at each state, which gives the routing path from source to destination.

The key difference from the compared method is that this approach does not just switch between fixed XY and YX behavior. It explores multiple possible paths and learns a better one under fault conditions.

6. How is the solution evaluated?

The solution is evaluated through **simulation**. The algorithm is implemented in **Python** and tested in **Jupyter Notebook** on a machine with an Intel i5 processor and 8 GB RAM.

The authors study several cases:

- fault-free mesh,
- mesh with link faults,
- mesh with router faults,
- mesh with both link and router faults.

They also test different mesh sizes:

- **5×5**
- **10×10**
- **15×15**

For each mesh size, they use a fixed set of one hundred random source-destination pairs and compare the proposed RL-based routing algorithm against the routing method from reference [10].

7. What metrics are used for evaluation?

The main evaluation metrics are:

a. Packet delivery rate

They measure the **percentage of packets delivered** to the destination as the percentage of faulty links or faulty routers increases. This is shown in the plots on pages 5 and 6.

b. Average number of hops

They also measure the **average number of hops** taken by packets under different fault levels and mesh sizes. This is used to show whether the routing remains efficient even in faulty conditions.

c. Path optimality in case studies

In the specific examples from R1 to R24 in the 5×5 mesh, they compare how many hops are needed in fault-free and faulty cases. Their algorithm often keeps the path length equal to the fault-free shortest path, while the compared method needs more hops.

8. What are the limitations of this work?

A few limitations stand out.

a. It is only shown on mesh topology

The paper focuses only on **mesh-based NoC**. The authors themselves mention extending it to other topologies as future work.

b. Evaluation is fairly narrow

The evaluation is based on high-level simulation in Python, not hardware-level implementation or a detailed NoC simulator. So the results show routing behavior, but not full architectural cost.

c. Limited evaluation metrics

The study mainly evaluates:

- packet delivery,
- and hop count.

It does **not** deeply evaluate:

- latency,
- area overhead,
- power,
- hardware complexity,
- training overhead,
- or deadlock behavior. The conclusion explicitly mentions latency, area, and power as future work, which confirms they are not covered yet.

d. Fault model is somewhat abstract

The paper varies the number of faulty routers and links, but it does not give a very deep discussion of fault patterns, fault dynamics, or whether faults are permanent, intermittent, clustered, or time-varying.

e. Comparison baseline is limited

The proposed method is mainly compared against one approach from [10]. That gives a useful reference, but not a broad comparison against many state-of-the-art fault-tolerant routing methods.

9. What ideas for future work arise from this paper?

The paper itself gives some immediate future directions:

- extend the routing algorithm to **other NoC topologies**,
- investigate its effect on **latency**,
- study **area overhead**,
- and study **power impact**.

Other natural future directions from this paper are:

a. Use more advanced RL methods

This paper uses basic Q-learning. A natural next step would be testing more advanced RL methods, especially if the state space becomes large.

b. Study online adaptation

It would be interesting to see whether the routing policy can adapt online as faults appear during runtime instead of relying mainly on prior learning setup.

c. Add hardware-aware evaluation

A stronger follow-up would examine implementation cost inside routers and whether the method remains practical on real NoC hardware.

d. Compare against more routing algorithms

A broader comparison with other fault-tolerant and adaptive routing algorithms would make the claims stronger.

Overall takeaway

Yes, this is clearly a **NoC routing paper**. The paper solves the problem of **fault-tolerant packet routing in mesh-based NoC using reinforcement learning**. Its main contribution is using **Q-learning** to find routing paths under link and router faults, and the results show better packet delivery and lower average hop count than the compared method in many cases. The strongest limitation is that the work stays at a fairly high simulation level and does not yet study latency, area, and power in detail.

January 30, 2026

Reinforcement Learning Based Fault-Tolerant Routing Algorithm for Mesh Based NoC and Its FPGA Implementation

1. Is the paper related to routing algorithm in NoC?

Yes. This paper is directly about a **routing algorithm in NoC**. Its main contribution is a **reinforcement learning based fault-tolerant routing algorithm**, called **RL-FTR**, for **mesh-based NoC**, along with an **FPGA implementation.**

2. What problem is being solved?

The paper solves the problem of **routing packets correctly and efficiently in a mesh NoC when links and routers are faulty.** The authors aim to improve:

- **fault tolerance**,
- **packet delivery**,
- and **routing efficiency**

without relying only on rigid, predefined routing rules. They want the algorithm to find an **optimal routing path** between source and destination even when the mesh contains faulty links, faulty routers, or both.

3. Why is the problem interesting in the first place?

The problem is interesting because NoC is a key communication structure in MPSoCs and many-core systems, and its usefulness depends heavily on reliable packet routing. As transistor sizes shrink and switching activity rises, routers and links are more likely to fail because of heat and stress. When that happens, communication quality drops, which directly hurts system reliability and performance. Since NoC is used in real-time and safety-critical systems too, fault-tolerant routing becomes an important practical issue.

The paper also argues that traditional routing rules are too rigid. They work for common cases, but when new routing situations appear, they need manual redesign. That makes learning-based routing attractive.

4. Why is the problem challenging to solve?

The problem is difficult because several things have to be handled at once.

First, faults can occur in **both routers and links**, and either type of fault can block otherwise valid paths.

Second, fixed routing schemes are often not flexible enough. They usually follow predefined rules and may keep sending packets through the same nodes, which can lead to congestion, queuing, or inefficient rerouting under faults.

Third, using reinforcement learning in a large routing problem creates scaling issues. The paper specifically points out that standard Q-table based routing can suffer from:

- **outdated Q-values**,
- **large memory needs**,
- and **slow exploration** in larger systems.

So the challenge is not only to find a valid path under faults, but to do it in a way that is scalable enough for practical NoC use.

5. What key idea(s) are being exploited to solve this problem?

The paper uses a few main ideas.

a. Model routing as a reinforcement learning problem

The routing problem is modeled as an **MDP**:

- routers are treated as **states**,
- links are treated as **actions**,
- moving to a neighbor router is a **state transition**,
- and rewards are assigned according to whether an action leads toward a valid path or the destination.

b. Use multi-agent reinforcement learning

Instead of a single learning agent, the paper uses **decentralized MARL**, where each router acts as an **independent learner**. The routers cooperate toward the common goal of delivering packets along the shortest path. This is meant to make the approach more suitable for large interconnect networks.

c. Introduce a two-level Q-table

This is one of the main technical ideas. The authors say the standard Q-table in Q-routing scales poorly. Their **two-level Q-table**

groups routers by row and stores port-based Q-values more compactly. They claim this gives more learning information while using about half the memory of the Q-routing style table and improves convergence behavior.

d. Use learned Q-values for routing

Once training is done, the router chooses the port with the highest Q-value toward the destination group, and repeats this step hop by hop until the destination is reached. Because the agent has explored the full topology during learning, the routing path can adapt to faults.

e. Claim deadlock and livelock freedom

The paper states that since RL-FTR has control over the complete routing path from source to destination, the proposed algorithm is free from **deadlock** and **livelock**.

6. How is the solution evaluated?

The solution is evaluated in **two ways**.

a. Software simulation

The RL-FTR algorithm is implemented in **System-C** and integrated into a **cycle-accurate NoC simulator**. The experiments test different mesh sizes and increasing percentages of faulty links, faulty routers, and combined faults.

The simulator setup uses:

- synthetic traffic,
- injection rate of 0.02 packets/cycle/IP,
- simulation time of 2,00,000 cycles,
- wormhole switching,
- 32-bit flits,
- 64-flit packet size.

b. FPGA implementation

The real-time functionality is then tested by implementing the router logic with RL-FTR on a **Xilinx Kintex-7 KC705 FPGA**. The authors modify the routing logic of an existing NoC router architecture and use **Vivado** plus **VIO** to drive and observe signals.

They also study specific case examples in a **4×4 mesh**, including:

- fault-free case,
- link fault case,
- router fault case,
- and mixed link/router fault case.

7. What metrics are used for evaluation?

The main evaluation metrics are:

a. Percentage of packets delivered

This is one of the main metrics in the simulator results. The paper measures how packet delivery changes as the percentage of faulty links, faulty routers, or both increases. The plots on pages 6 to 8 show this for several mesh sizes.

b. Average network latency

The simulator also reports **average network latency** under increasing fault rates. This is used to compare RL-FTR against the reference algorithms [7], [8], and [9].

c. Routing path length / number of hops

In the FPGA case studies, the paper compares the routing paths found by the different algorithms and reports whether RL-FTR reaches the destination in fewer hops. For example, in several faulty cases RL-FTR keeps the hop count equal to the fault-free case, while some comparison methods need more hops.

d. Time taken for packet travel in FPGA case studies

For the FPGA-based experiments, the authors compare the actual time taken for a packet to travel from router R1 to router R15 under different fault conditions. Table 2 summarizes this.

e. Hardware resource utilization

The FPGA implementation reports resource usage such as:

- LUTs,
- registers,
- multiplexers,
- and BUFGs.

f. On-chip power consumption

The paper also reports dynamic, static, and total power consumption from the FPGA implementation.

8. What are the limitations of this work?

A few limitations stand out.

a. Only mesh topology is considered

The paper is restricted to **mesh-based NoC**. The conclusion explicitly says extension to other topologies is future work.

b. The experiments use synthetic traffic

The System-C experiments are done using **synthetic traffic**, not a broad workload set of realistic application traces. That limits how much can be said about behavior under more realistic traffic patterns.

c. FPGA study is limited to a 4×4 mesh

The real hardware demonstration is useful, but it is relatively small. The FPGA case studies focus on a **4×4 mesh**, so the hardware validation of scalability is limited.

d. Learning overhead is not deeply analyzed

The paper explains the Q-table update and memory improvement, but it does not deeply quantify:

- training time overhead,
- convergence cost,
- update frequency in runtime conditions,
- or how the learning behaves under rapidly changing faults.

e. Deadlock/livelock claim is not formally proved

The paper states that RL-FTR is free from deadlock and livelock because it has control over the full routing path, but it does not present a detailed formal proof like many routing-theory papers do.

f. Fault model is still fairly abstract

Although the paper studies faulty links and routers, it does not deeply explore different fault dynamics such as intermittent faults,

temporal evolution of faults, or correlated failures.

9. What ideas for future work arise from this paper?

The paper itself suggests:

- extending the algorithm to **other topologies**,
- and implementing those on **FPGA** as well.

Other natural future directions are:

a. Use realistic application traffic

Testing RL-FTR under realistic workloads instead of only synthetic traffic would make the results stronger.

b. Study larger hardware implementations

A larger FPGA or ASIC-style implementation would help confirm scalability of the two-level Q-table and decentralized MARL approach.

c. Explore dynamic and intermittent faults

Since many NoC reliability problems are not just permanent faults, it would be useful to test how RL-FTR handles changing fault conditions over time.

d. Compare against more recent RL-based methods

The paper compares against three algorithms, including one earlier RL-based approach. Future work could compare RL-FTR against newer DRL-based or adaptive NoC routing methods.

e. Deeper analysis of learning cost

A useful next step would be to quantify:

- learning convergence time,
- storage overhead for larger systems,
- and runtime update cost.

Overall takeaway

Yes, this paper is directly about a **NoC routing algorithm**. It proposes **RL-FTR**, a **reinforcement learning based fault-tolerant**

routing algorithm for mesh NoC, and strengthens the earlier idea by adding:

- **decentralized multi-agent RL**,
- a **two-level Q-table** for better scalability,
- **System-C cycle-accurate evaluation**,
- and **FPGA implementation.**

Its strongest contribution is that it goes beyond high-level routing simulation and includes real hardware validation, while still showing better packet delivery and competitive latency under faulty conditions. Its main weaknesses are that it remains limited to mesh topology, uses synthetic traffic in simulation, and does not deeply analyze learning overhead or formally prove some routing properties.

February 2, 2026

LARE: A Linear Approximate Reinforcement Learning Based Adaptive Routing for Network-on-Chips

1. Is the paper related to routing algorithm in NoC?

Yes. This paper is directly about a **routing algorithm in NoC**. Its main contribution is an **adaptive routing method** based on **reinforcement learning**, and then a lower-overhead version called **LARE** that uses **linear function approximation** instead of a full RL state-action table.

2. What problem is being solved?

The paper tries to solve the problem of **how to design an adaptive NoC routing algorithm that handles complex, time-varying traffic better than static routing methods while keeping hardware overhead practical**. The authors argue that existing routing algorithms often fail to balance traffic well under diverse workloads because their routing logic is designed in a static way. They want a routing algorithm that can:

- use network state information,
- choose better paths dynamically,
- reduce communication latency,
- and improve load balance.

At the same time, they want to avoid the large hardware cost of a straightforward RL implementation.

3. Why is the problem interesting in the first place?

The problem is interesting because routing has a large effect on **latency** and **throughput** in NoCs, and modern NoCs are increasingly exposed to:

- diverse application mixes,
- bursty traffic,
- and time-varying communication patterns.

Under those conditions, a routing strategy that is fixed in advance may not adapt well. The paper’s basic motivation is that future many-core systems need routing that reacts to traffic conditions more intelligently than classical static rules. Since RL can adapt policy based on experience, it becomes an interesting tool for this problem.

4. Why is the problem challenging to solve?

The problem is challenging for two main reasons.

First, routing in NoC becomes a **multi-objective optimization problem** under dynamic traffic. The algorithm should not just minimize one metric. It should also consider:

- latency,
- load balancing,
- congestion behavior,
- and long-term routing effects.

Second, while RL may improve routing quality, **hardware implementation overhead** becomes a major challenge. The paper emphasizes that if a full RL state-action table is used, the cost grows quickly with network size. This is essentially a scalability problem, and the authors call out the “curse of dimensionality” in the routing state space.

So the hard part is not just using RL to improve routing, but doing it in a way that stays practical for hardware.

5. What key idea(s) are being exploited to solve this problem?

The paper uses a sequence of ideas.

a. Use RL to make routing decisions

The first idea is to view routing as an RL problem. Each router contains an RL agent that observes network state and chooses an output port. The RL agent learns which action is best by maximizing long-term cumulative reward.

b. Use a reward function that combines multiple routing goals

The reward is designed to encourage:

- lower average latency,
- better buffer load balance,
- and isolation of packets with the same destination when endpoint congestion appears.

The reward function includes latency (L), number of idle virtual channels (U), and a measure (W) for packets with the same destination per VC. This is how the RL model captures multiple objectives at once.

c. Restrict actions to minimal-path choices

For each router, the action space is kept simple: the RL agent chooses between two minimal-path directions, (a_x) and (a_y). This keeps the decision space manageable.

d. Replace the full RL Q-table with a linear approximation

This is the core contribution. Instead of storing a large state-action table, **LARE** approximates the Q-value with a **linear function** over a small set of selected features. This reduces storage cost and hardware complexity.

e. Use carefully chosen routing features

LARE uses five features in the linear model:

- constant term,

- destination router ID,
- input port,
- distance already traversed,
- remaining distance to destination.

These features are used to estimate the Q-value of each feasible output port. The router then chooses the port with the highest estimated Q-value, with a small probability of random exploration.

f. Use escape virtual channels for deadlock avoidance

Because generic shortest-path adaptive routing can deadlock, the paper uses an **escape VC mechanism**, following Duato-style deadlock avoidance.

6. How is the solution evaluated?

The solution is evaluated through **simulation** and **area synthesis**.

a. Full-system simulation

The authors use **gem5** with **Garnet 2.0** for full-system, cycle-accurate evaluation. The simulated platform is:

- 64 CPU cores,
- connected by an **8×8 mesh NoC**,
- with private L1 caches and shared L2 cache.

They compare:

- **DOR**,
- **O1Turn**,
- **Footprint**,
- **full RL-based routing**,
- and **LARE**.

b. Synthetic traffic evaluation

They test under:

- uniform random,
- shuffle,
- transpose,
- and a mixed synthetic traffic pattern.

c. Real application evaluation

They also run **11 applications from the PARSEC 2.1 suite** in gem5 full-system mode and compare normalized average packet latency across routing algorithms.

d. Hardware area synthesis

To evaluate implementation overhead, they use **Synopsys Design Compiler** with **90 nm TSMC technology** and compare the router area of:

- a baseline router,
- Footprint,
- full RL routing,
- and LARE.

7. What metrics are used for evaluation?

The main evaluation metrics are:

a. Average packet latency

This is the main performance metric in both synthetic traffic and real applications. Figures 2 and 3 on page 4 focus on average packet latency.

b. Saturation behavior / latency-throughput curve

Under synthetic traffic, they compare how average latency changes with increasing **injection rate**, which shows the effective throughput and saturation point of the routing algorithms.

c. Normalized average packet latency under real workloads

For PARSEC workloads, all results are normalized to DOR. This lets them compare routing quality under practical applications.

d. Router area overhead

For hardware cost, they report synthesized **area in mm^2** for the different routing implementations. Table I on page 4 shows this clearly.

8. What are the limitations of this work?

A few limitations are visible.

a. It is mainly focused on performance and area

The evaluation emphasizes:

- latency,
- and area.

It does not deeply study:

- power,
- energy,
- fault tolerance,
- or thermal/reliability effects.

b. The method is sensitive to parameter tuning

The paper itself says that LARE is **susceptible to some parameters**, and these parameters must be continuously tuned to get the best performance. That means the method is not fully plug-and-play.

c. The linear approximation may limit expressiveness

LARE reduces hardware cost by using a linear model, but a linear model is still a simplification. It may not capture all complex routing behaviors as fully as a richer RL model could. The paper shows that LARE is close to full RL, but this is still a tradeoff.

d. Action space is restricted

The RL agent chooses only among two minimal directions. That keeps things manageable, but it also limits how much flexibility the routing policy has compared to a broader adaptive search.

e. No hardware implementation beyond synthesis

The paper gives synthesis-based area results, but not a full FPGA or silicon implementation. So implementation practicality is estimated, not fully demonstrated in real hardware.

9. What ideas for future work arise from this paper?

The paper directly mentions:

- **parameter optimization**,
- and training on **more traffic patterns**.

Other natural future directions are:

a. Explore stronger approximation models

Since LARE uses a linear approximation, a next step could be testing richer approximators that still remain hardware-efficient.

b. Broaden the objective set

The reward currently targets latency, load balance, and isolation. Future work could include energy, power, or reliability in the reward.

c. Study more realistic deployment questions

For example:

- online retraining,
- adaptation under workload phase changes,
- sensitivity to system size,
- and implementation on FPGA or ASIC for stronger hardware validation.

d. Extend to different NoC topologies

The work is evaluated on an 8×8 mesh. A useful next step would be checking whether the same method works well on torus, irregular, or hierarchical NoCs.

Overall takeaway

Yes, this is clearly a **NoC routing paper**. The central problem is **how to make adaptive routing in NoCs more intelligent under complex traffic without paying the full hardware cost of a standard RL implementation**. The main contribution is **LARE**, which uses a **linear approximation of RL Q-values** to preserve most of the performance benefit of RL while sharply lowering area overhead. Its strongest point is the balance it achieves between **routing performance** and **implementation cost**. Its main weakness is that it still depends on parameter tuning and is evaluated mainly for latency and area, not a broader range of system concerns.

February 11, 2026

Understanding MTTF in Routers

I am reading the paper: [Clotho: Proactive wearout deceleration in Chip-Multiprocessor interconnects]

Important Ideas

- Introduction: Current state → Problem → Why this (use examples) → Touch on prior work → Proposed methodology
-
- Only considers HCI and EM
  - Mentions that NBTI is caused by underutilization and can be mitigated by a technique known as *Use it or Lose it*
- Severity of a core failing is far less than a NoC component failing like routers or links
- Need proactive techniques
  - *Predicting failure* and routing to avoid failures
- Primary cause of failure:
  - probability of bit transition in wire or transistor
  - temperature (both HCI and EM)
  - PV (only HCI)
- Temperature is relatively constant in a certain region but can vary between regions
- Router uses minimal power and hence dissipates less heat
  - Core generates more heat and increase temperature of the router
- Threshold voltage and effective gate length are more susceptible to PV
-

Claude's Interpretation

1. Is the paper related to routing in NoC?
   Yes, absolutely. The paper presents Clotho, a wearout-aware adaptive routing algorithm specifically designed for Network-on-Chip (NoC) interconnects in Chip-Multiprocessors (CMPs).
2. What problem is being solved?
   The paper addresses prolonged operational stress-induced wearout in NoC components, specifically from Electromigration (EM) and Hot-Carrier Injection (HCI). Unlike core failures which can be managed through redundancy, a single NoC component failure can render an entire CMP unusable through deadlock, network segmentation, or peripheral disconnection.

3. Why is the problem interesting in the first place?
   Several factors make this compelling:

Asymmetric criticality: NoC failures are catastrophic compared to core failures (Fig. 1 scenarios illustrate this well)
Scaling trajectory: ITRS projects a 10× increase in wear rate over the next decade
Architectural vulnerability: The interconnect lacks the redundancy that cores enjoy
Performance-reliability tension: High utilization (needed for performance) accelerates wearout through increased switching activity

4. Why is the problem challenging to solve?
   Key challenges include:

Multi-factor complexity: Wearout depends on switching activity, temperature, process variation, and accumulated wear—all varying spatially and temporally
Different failure mechanisms: EM (affects links/wires) and HCI (affects router transistors) have different sensitivities and acceleration factors
Global optimization requirement: Local decisions can lead to suboptimal global lifetime outcomes
Unknown constants: Process-specific parameters in MTTF equations are proprietary, making absolute lifetime prediction difficult
Real-time constraints: Routing decisions must be made quickly despite complex lifetime calculations

5. What key idea(s) are being exploited to solve this problem?
   The core insights are:
   Proactive vs. Reactive: Unlike fault-tolerant routing that reacts to failures, Clotho prevents failures by steering traffic away from vulnerable components before they fail.
   Multi-metric awareness: Combines four inputs:

Dynamic switching activity (measured via utilization counters)
Spatial temperature variation (sampled every 10^7 cycles)
Static process variation (measured post-fabrication, stored in ROM)
Accumulated wear-to-date

Unified wear model: Equation (17) creates a combined routing metric that weights both EM and HCI lifetime contributions, selecting paths with maximum cumulative lifetime (Wnorm).
Global information sharing: Piggybacks wear-related metrics in

packet header flits (borrowing from GCA routing), enabling source routers to have global visibility for path selection.
Exponential weighting: The coefficient functions (Eq. 18) exponentially penalize paths near their MTTF limits, making the routing strongly avoid "weakest links."
6. How is the solution evaluated?
The evaluation uses:

Platform: gem5 full-system simulator with Ruby memory model and Garnet NoC simulator
System: 64-core (8×8 mesh) x86 CMP with MESI cache coherence
Workloads: PARSEC benchmark suite applications
Methodology: Each simulation repeated 15 times with different temperature mappings, results averaged
Comparison points:

Baseline: Dimension-Order Routing (DOR)
Clotho-LAR (Locally-Adaptive Routing)
Clotho-GAR (Globally-Adaptive Routing)

Sensitivity analysis: Varied EM vs. HCI weighting ratios (0-100%, 25-75%, 50-50%, 75-25%, 100-0%)

7. What metrics are used for evaluation?
   Primary metrics:

Acceleration Factor (AF): Ratio of MTTF under Clotho to MTTF under baseline DOR (analogous to speedup in performance domain)

AF_EM: Clotho-GAR achieves 66% average increase
AF_HCI: Clotho-GAR achieves 8% average increase

Performance impact: Execution time in cycles (Clotho-GAR shows ~2% degradation on average)
Bottom 10% MTTF cases: Focus on worst-case reliability scenarios (most likely to fail first)

8. What are the limitations of this work?
   Several notable limitations:
   Limited HCI improvement: Only 8% average increase in MTTF_HCI vs. 66% for EM, because:

HCI model is less sensitive to input parameter variations
Only header flits traverse router critical paths (HCI), while all flits cross links (EM)

Proprietary constants problem: Cannot determine absolute lifetime or optimal EM/HCI weighting ratio—must assume 50-50 split without foundry data
Hardware overhead: ~5% router area overhead for routing maps, lifetime tables, and computation logic
LAR ineffectiveness: Locally-adaptive version shows minimal benefit (3% EM improvement, -5% HCI degradation), suggesting global information is essential
NBTI exclusion: Paper focuses only on EM and HCI, explicitly excluding NBTI (though they cite "Use it or Lose it" for NBTI mitigation)
Sampling granularity: Temperature sampled every 10^7 cycles, utilization every 10^4 cycles—may miss transient hotspots
Coarse spatial resolution: Temperature and PV modeled at tile/router granularity, not finer-grained within routers
9. What ideas for future work arise from this paper?
Integration with NBTI mitigation: Combining Clotho with techniques like "Use it or Lose it" to address all three major wearout mechanisms simultaneously.
Adaptive weighting: Developing runtime mechanisms to adjust EM/HCI weighting ratios based on observed wear rates rather than fixed 50-50 split.
Machine learning approaches: Using RL (like your thesis!) to learn optimal routing policies that maximize lifetime without explicit MTTF models—could handle unknown proprietary constants better.
Hierarchical/hybrid approaches: Combining global and local information more intelligently—perhaps GAR for strategic decisions, LAR for tactical adjustments.
Wear-aware task mapping: Co-optimizing application mapping to cores and routing to minimize total system wear.
Finer-grained monitoring: Investigating whether more frequent sampling or spatial resolution improves lifetime extension despite energy costs.
Graceful degradation: Combining proactive (Clotho) with reactive (fault-tolerant routing) approaches—use Clotho to delay failures, then gracefully handle inevitable failures when they occur.
Multi-objective optimization: Explicitly balancing performance, power, and lifetime rather than treating performance as a constraint.

February 14, 2026

Understanding Aging Models in Clotho

Today, I focused on understanding the theoretical background behind the aging models used in the Clotho paper, since my work will build directly on it. Before moving further into implementation, it felt necessary to clearly understand how aging is modeled and what assumptions are made.

From reviewing the material, I noticed that most NoC reliability papers, including Clotho, primarily focus on **Hot Carrier Injection (HCI)** and **Electromigration (EM)**, while largely ignoring **Bias Temperature Instability (BTI)**. Based on this observation, I decided to follow the same approach for now and focus on HCI and EM. BTI can be added later if needed, since it appears to be a relatively straightforward extension compared to the complexity already present in HCI and EM modeling.

Section 1: Temperature and Process Variation

Temperature

Temperature is not uniform across the chip. Each router tile has its own temperature driven by the computational load of its associated core.

Key properties:

- Range: 65°C to 85°C (338K to 358K)
- Updated every `10^7` cycles to mimic real thermal dynamics
- Spatially correlated, so nearby tiles tend to have similar temperatures
- The NoC itself contributes only about 10% of chip power, so routers do not meaningfully self-heat. Temperature is mainly driven by the cores.

In the implementation, this can be modeled as a synthetic per-router temperature map stored in `GarnetNetwork`, updated periodically with spatially correlated random values. No real thermal simulator is needed. This is exactly what Clotho does.

Process Variation (PV)

PV is fabrication-time imprecision that causes transistor parameters to deviate from their nominal values. It affects two parameters per router:

Threshold voltage:

$$V_{th} = V_{th0} + \Delta V_{th}, \quad V_{th0} = 179\ \mathrm{mV}$$

Effective gate length:

$$L_{eff} = L_{eff0} + \Delta L_{eff}, \quad L_{eff0} = 14.4\,\text{nm}$$

Each deviation has two components:

$$\Delta V_{th} = \Delta V_{th,rand} + \Delta V_{th,sys}$$

$$\Delta L_{eff} = \Delta L_{eff,rand} + \Delta L_{eff,sys}$$

Systematic variation is spatially correlated, meaning nearby routers deviate similarly. It is modeled using a spherical correlation function:

$$\rho(r) = \begin{cases} 1 - \frac{3r}{2\phi} + \frac{r^3}{2\phi^3}, & r \le \phi \\ 0, & r > \phi \end{cases}$$

where:

$$\phi = 0.5$$

Here, ϕ is expressed as a fraction of die length, so routers more than half a die apart are uncorrelated.

Random variation has no spatial correlation and is purely per-transistor noise.

Combined standard deviation:

$$\sigma_{total} = \sqrt{\sigma_{rand}^2 + \sigma_{sys}^2}$$

Parameter values from the paper:

Parameter	σ_{rand}	σ_{sys}	ϕ
V_{th}	6.3%	6.3%	0.5
L_{eff}	3.2%	3.2%	0.5

Key point: PV is generated once at initialization and never changes. It is a fabrication constant. Per-router V_{th} and L_{eff} should be stored in `GarnetNetwork` during initialization.

Precomputing I_{sub} From PV

Since PV is static and all other parameters in this chain are technology constants, I_{sub} can be fully precomputed per router at initialization.

Step 1: Compute V_{dsat}

$$V_{dsat}(V_{th}, L_{eff}) = \frac{(V_{gs} - V_{th}) \cdot L_{eff} \cdot E_{cr}}{(V_{gs} - V_{th}) + L_{eff} \cdot E_{cr}}$$

Step 2: Compute E_m

$$E_m(V_{dsat}) = \frac{V_{ds} - V_{dsat}}{\sqrt{3t_{ox}x_j}}$$

Step 3: Compute I_{sub}

$$I_{sub} = C_2 \cdot I_{ds} \cdot \exp\left(\frac{-\phi_i}{q\lambda_e E_m}\right)$$

All parameters except V_{th} and L_{eff} are technology node constants set once from literature. The resulting I_{sub} can be stored as a single float per router, and this chain does not need to run again.

Section 2: Failure Models

Electromigration (EM) — Per Link

EM is metal ion drift in interconnect wires under high current density. It affects the `Vdd` and `GND` connections to long router-to-router link drivers specifically, because long wires have large capacitance and therefore high charge and discharge currents.

Final model:

$$MTTF_{EM} \propto \alpha_{link}^{-2} \cdot \exp\left(\frac{E_{a,EM}}{kT}\right)$$

Inputs:

- $\alpha_{link} = 0.5 \times U_{link}$
  Link switching activity derived from utilization
- T
  Per-tile temperature in Kelvin
- $E_{a,EM}$
  Published activation energy, about 0.9 eV
- k
  Boltzmann's constant

Output:

- Proportional MTTF value per link. Higher means longer remaining lifetime.

Key sensitivity:

- Quadratic in switching activity, α^{-2}
- Doubling link utilization quarters EM lifetime

This is why EM sees much larger improvement under Clotho than HCI. EM is much more sensitive to load redistribution.

Hot-Carrier Injection with PV (HCI+PV) — Per Router

HCI degrades transistors by trapping hot carriers in the gate oxide, permanently shifting V_{th} upward and slowing the critical path until timing violations occur.

Final model:

$$MTTF_{HCI,PV} \propto \frac{(I_{sub}(V_{th}, L_{eff}))^{-N}}{\alpha_{router}} \cdot \exp\left(\frac{E_{a,HCI}}{kT}\right)$$

Inputs:

- $\alpha_{router} = 0.1 \times U_{router}$
  Critical path switching activity. Only header flits exercise the critical path.
- T
  Per-tile temperature in Kelvin
- I_{sub}
  Precomputed per router from PV at initialization
- N
  Empirical exponent, about 1.5
- $E_{a,HCI}$
  Published activation energy, about 0.2 to 0.4 eV

Output:

- Proportional MTTF value per router. Higher means longer remaining lifetime.

Key sensitivity:

- Linear in switching activity, α^{-1}
- Less sensitive than EM to load redistribution

PV creates a fixed baseline shift, so routers with unfavorable V_{th} or L_{eff} begin with reduced timing margin even before any aging occurs.

What You Can Do With These Models

During routing decisions

- Compute proportional MTTF for each candidate next-hop router and link
- Normalize relative to current network state
- Use the values to score routing options
- Prefer paths through high-MTTF components

During simulation

- Track per-component MTTF evolution as utilization accumulates
- Update periodically as utilization snapshots and the temperature map refresh

At the end of simulation

- Collect final per-component MTTF values for all routers and links
- Define system lifetime as the minimum MTTF across all components
- Report the bottom 10% of the MTTF distribution, since the weakest components determine system fate
- Compute acceleration factor relative to the DOR baseline

Acceleration Factor

$$AF_{EM} = \frac{MTTF_{EM}|_{RL}}{MTTF_{EM}|_{DOR}}$$

$$AF_{HCI} = \frac{MTTF_{HCI,PV}|_{RL}}{MTTF_{HCI,PV}|_{DOR}}$$

Unknown constants and I_{sub} cancel in the ratio because PV is identical across both runs. Only E_a values from the literature, utilization, and temperature from simulation are needed.

February 15, 2026

Choosing the Simulation Platform

Today, I looked into the simulators used in the papers I have been reading, since I need to decide what environment I will use for my own implementation and evaluation.

From the papers so far, it seems that two simulators appear repeatedly: **BookSim2** and **gem5**. At this stage, I am trying to understand the role of each one and why different papers choose one over the other. My initial impression is that BookSim2 is often used

for network-level studies because it is simpler and more focused on NoC behavior, while gem5 is used more often when the evaluation needs to be closer to a complete system.

Most of the papers I have reviewed use **gem5**, especially when they want to evaluate routing methods under more realistic workloads and system conditions. Since my work will likely need to connect routing behavior with reliability and long-term effects rather than only idealized network traffic, gem5 seems like the more suitable choice.

Based on this, I decided that I will try to use **gem5** as the main simulator for my work. This seems to be the better option for staying aligned with the literature and for producing results that are easier to compare with related papers. My next step will be to understand how the NoC is modeled inside gem5, how routing is implemented, and what modifications will be needed to integrate the kind of aging-aware routing I want to study.

I used Claude Code to understand the gem5 repo quickly.
Important Links:
gem5 repo: https://github.com/gem5/gem5
gem5: https://www.gem5.org/
gem5 interconnect: https://www.gem5.org/documentation/general_docs/ruby/interconnection-network/
gem5 Garnet: https://www.gem5.org/documentation/general_docs/ruby/garnet-2/
gem5 Garnet Standalone: https://www.gem5.org/documentation/general_docs/ruby/Garnet_standalone/
gem5 Garnet Synthetic Traffic: https://www.gem5.org/documentation/general_docs/ruby/garnet_synthetic_traffic/

Since the MTTF models only need link and router utilization and temperature of the router and link, I need to figure out how gem5/Garnet handles it.

Understanding gem5 / Garnet2.0 Simulation Internals

1. Network Traversal Lifecycle — Code-Level Walkthrough

The gem5 website description maps directly to this codebase as follows:

NI → Router entry (NetworkInterface.cc)

The NI converts protocol messages into flits and injects them onto the network via `NetworkLink`. Each NI manages credit-based flow control with its attached router's Local input port.

Link traversal (NetworkLink.cc:59-70)

```C++
void NetworkLink::wakeup() {
    if (link_srcQueue->isReady(curCycle())) {
        flit *t_flit = link_srcQueue->getTopFlit();
        t_flit->set_time(curCycle() + m_latency);
        linkBuffer.insert(t_flit);
        link_consumer->scheduleEventAbsolute(clockEdge(m_latency));
        m_link_utilized++;           // ⟵ link utilization counter
        m_vc_load[t_flit->get_vc()]++; // ⟵ per-VC load counter
    }
}
```

Each time a flit traverses a link, `m_link_utilized` is incremented. This is the raw link utilization counter.

Router pipeline (Router.cc:63-87)

```C++
wakeup()
  → InputUnit::wakeup() × all inports
  → OutputUnit::wakeup() × all outports (credit updates from downstream)
  → SwitchAllocator::wakeup() (SA-I then SA-II)
  → CrossbarSwitch::wakeup() (winner flits forwarded to output links)
```

Route computation (InputUnit.cc → RoutingUnit.cc:136-176)

HEAD / HEAD_TAIL flits trigger `outportCompute()` in the RoutingUnit. This is the single decision point where the routing algorithm runs.

---

2. Router and Link Utilization — What Already Exists

Link Utilization

`NetworkLink` tracks utilization natively:

- `m_link_utilized` (NetworkLink.hh:82): raw flit count per link
- `m_vc_load[]` (NetworkLink.hh:83): per-VC load vector

Access via:

- `getLinkUtilization()`
- `getVcLoad()`

`GarnetNetwork::collateStats()` (GarnetNetwork.cc:386-415) loops over all `m_networklinks[]` and aggregates:

- `EXT_IN_`
- `EXT_OUT_`
- `INT_` utilization

Router Utilization (per-router)

Tracked via `collateStats()` (Router.cc:200-213):

- `m_buffer_reads` / `m_buffer_writes`
- `m_crossbar_activity`
- `m_sw_input_arbiter_activity`
- `m_sw_output_arbiter_activity`

`m_crossbar_activity` is the most direct proxy for router utilization.

Active VC Count

Already used in adaptive routing:

- `Router::get_num_active_vcs(outport, vnet)`
- `Router::has_free_vc(outport, vnet)`

These provide congestion signals.

---

3. How to Access Neighbouring Router's Utilization

Each router has a pointer to `GarnetNetwork` via `m_network_ptr`.

std::vector<Router > *m_routers;*
std::vector<NetworkLink > m_networklinks;

Option A — Direct Router Query

Inside `RoutingUnit::outportComputeCustom()`:

```C++
GarnetNetwork* net = m_router->get_net_ptr();
int neighbor_id = ...;
Router* neighbor = net->getRouter(neighbor_id);
int neighbor_crossbar_activity = neighbor->getCrossbarActivity();
```

Requirements:

- Add `getRouter(int id)` to expose `m_routers`
- Add runtime getter for `m_crossbar_activity`

Option B — OutputUnit Credit Count

Already available:

- `OutputUnit::get_credit_count(vc)`

Low credit → congested downstream router

February 16, 2026

How to implement custom RL routing algorithm in gem5

1. Custom Routing Algorithm — Implementation Hook

Defined in:

```
CommonTypes.hh:43
enum RoutingAlgorithm { TABLE=0, XY=1, RANDOMOBLIVIOUS=2,
WESTFIRST=3, CUSTOM_=4, ... };
```

RoutingUnit.cc:

```
case CUSTOM_:
outport = outportComputeCustom(route, inport, inport_dirn);
break;
```

Stub:

```
int RoutingUnit::outportComputeCustom(RouteInfo route, int inport,
PortDirection inport_dirn)
{
panic("%s placeholder executed", FUNCTION);
}
```

Activated via:

--routing-algorithm=4

Inputs available:

- `route.src_router`
- `route.dest_router`
- `route.hops_traversed`
- `inport_dirn`

Access to:

- `m_router`
- Entire `GarnetNetwork`

---

2. Global RL Routing Agent Architecture

Single agent with full network visibility.

Implementation:

- Store Q-table / model in `GarnetNetwork`
- All routers query it during `outportComputeCustom()`

State

- For all routers: `crossbar_activity[i]`, `num_active_vcs`
- For all links: `link_utilized[l]`
- Current router ID
- Destination router ID

Action

- Output port direction (N, S, E, W)

Reward

- Packet latency (`m_flit_network_latency`)
- Collected at NI

Challenge

`outportComputeCustom()` runs in critical path → expensive to call external code (e.g., Python)

---

3. Local RL Routing Agent Architecture

Each router is its own agent.

Storage

```
std::map<std::pair<int,int>, float> q_table;
```

State (local)

- `crossbar_activity`
- Active VCs per output port
- Credit count per port
- Relative position to destination

Action

- Choose output port among minimal candidates

Available signals (no new C++ needed)

Signal	Source
Active VCs	`get_num_active_vcs`
Free VC	`has_free_vc`
VC active time	`get_vc_active_time`
Credit count	`get_credit_count`
Link utilization	`getLinkUtilization`

Key property

- No global synchronization
- More realistic for hardware
- Simulation-friendly

---

4. Summary — Required Modifications

Minimal required changes:

1. Add:

```
Router* getRouter(int id);
```

2. Add:

int getCrossbarActivity();

3. Implement:

outportComputeCustom()

4. Decide update strategy:

- In-simulation Q-learning (C++)
- Episode-based updates (Python via `icn_gym.py`)

Reference:

- WestFirst adaptive routing in `RoutingUnit.cc` already uses VC-based congestion signals and can serve as a baseline.

February 19, 2025

Complete Rundown of my idea

Section 1: Temperature and Process Variation

Temperature

Temperature is **not uniform** across the chip. Each router tile has its own temperature driven by the computational load of its associated core. Key properties:

- Range: **65°C to 85°C** (338K to 358K)
- Updated every **10^7 cycles** to mimic real thermal dynamics
- Spatially correlated — nearby tiles tend to have similar temperatures
- The NoC itself contributes only ~10% of chip power so routers don't meaningfully self-heat — temperature is driven by the cores

In your implementation this is a **synthetic per-router temperature map** stored in GarnetNetwork, updated periodically with spatially correlated random values. No real thermal simulator needed — this is exactly what Clotho does.

Process Variation (PV)

PV is fabrication-time imprecision that causes transistor parameters to deviate from their nominal values. It affects two parameters per router:

Threshold voltage:

$$V_{th} = V_{th0} + \Delta V_{th}, \quad V_{th0} = 179\ \mathrm{mV}$$

Effective gate length:

$$L_{eff} = L_{eff0} + \Delta L_{eff}, \quad L_{eff0} = 14.4\ \mathrm{nm}$$

Each deviation has two components:

$$\Delta V_{th} = \Delta V_{th,rand} + \Delta V_{th,sys}$$

$$\Delta L_{eff} = \Delta L_{eff,rand} + \Delta L_{eff,sys}$$

Systematic variation is spatially correlated — nearby routers deviate similarly. Modeled using a spherical correlation function:

$$\rho(r) = 1 - \frac{3r}{2\phi} + \frac{r^3}{2\phi^3} \quad \text{for } r \le \phi, \quad \rho(r) = 0 \quad \text{for } r > \phi$$

Where $\phi = 0.5$ (as fraction of die length) — routers more than half a die apart are uncorrelated.

Random variation has no spatial correlation — purely per-transistor noise.

Combined standard deviation:

$$\sigma_{total} = \sqrt{\sigma_{rand}^2 + \sigma_{sys}^2}$$

Parameter values from the paper:

Parameter	σ_{rand}	σ_{sys}	ϕ
V_{th}	6.3%	6.3%	0.5
L_{eff}	3.2%	3.2%	0.5

Key point: PV is generated **once at initialization** and never changes — it's a fabrication constant. Store per-router V_{th} and L_{eff} in GarnetNetwork at init.

Precomputing I_sub From PV

Since PV is static and all other parameters in this chain are technology constants, I_{sub} can be fully precomputed per router at initialization:

Step 1 — V_dsat:

$$V_{dsat}(V_{th}, L_{eff}) = \frac{(V_{gs} - V_{th}) \cdot L_{eff} \cdot E_{cr}}{(V_{gs} - V_{th}) + L_{eff} \cdot E_{cr}}$$

Step 2 — E_m:

$$E_m(V_{dsat}) = \frac{V_{ds} - V_{dsat}}{\sqrt{3 t_{ox} x_j}}$$

Step 3 — I_sub:

$$I_{sub} = C_2 \cdot I_{ds} \cdot \exp\left(\frac{-\phi_i}{q \lambda_e E_m}\right)$$

All parameters except V_{th} and L_{eff} are technology node constants set once from literature. Store the resulting I_{sub} as a single float per router — the chain never needs to run again.

Section 2: Failure Models

Electromigration (EM) — Per Link

EM is metal ion drift in interconnect wires under high current density. Affects the **Vdd and GND connections** to long router-to-router link drivers specifically, because long wires have large capacitance and thus high charge/discharge currents.

Final model:

$$MTTF_{EM} \propto \alpha_{link}^{-2} \cdot \exp\left(\frac{E_{a,EM}}{kT}\right)$$

Inputs:

- $\alpha_{link} = 0.5 \times U_{link}$ — link switching activity derived from utilization
- T — per-tile temperature in Kelvin
- $E_{a,EM}$ — published activation energy (~0.9 eV)
- k — Boltzmann's constant

Output: Proportional MTTF value per link — higher means longer remaining lifetime.

Key sensitivity: Quadratic in switching activity (α^{-2}) — doubling link utilization **quarters** EM lifetime. This is why EM sees 66% improvement under Clotho while HCI only sees 8% — EM is far more sensitive to load redistribution.

Hot-Carrier Injection with PV (HCI+PV) — Per Router

HCI degrades transistors by trapping hot carriers in the gate oxide, permanently shifting V_{th} upward and slowing the critical path until timing violations occur.

Final model:

$$MTTF_{HCI,PV} \propto \frac{(I_{sub}(V_{th}, L_{eff}))^{-N}}{\alpha_{router}} \cdot \exp\left(\frac{E_{a,HCI}}{kT}\right)$$

Inputs:

- $\alpha_{router} = 0.1 \times U_{router}$ — critical path switching activity (only header flits exercise the critical path)
- T — per-tile temperature in Kelvin
- I_{sub} — precomputed per router from PV at initialization (static)
- N — empirical exponent (~1.5)
- $E_{a,HCI}$ — published activation energy (~0.2-0.4 eV)

Output: Proportional MTTF value per router — higher means longer remaining lifetime.

Key sensitivity: Linear in switching activity (α^{-1}) — less sensitive than EM to load redistribution. PV creates a fixed baseline shift — routers with unfavorable V_{th} or L_{eff} start with reduced timing margin before any aging begins.

What You Can Do With These Models

During routing decisions:

- Compute proportional MTTF for each candidate next-hop router and link
- Normalize relative to current network state
- Use to score routing options — prefer paths through high-MTTF components

During simulation (ongoing):

- Track per-component MTTF evolution as utilization accumulates
- Update periodically as utilization snapshots and temperature map refresh

At end of simulation (evaluation):

- Collect final per-component MTTF values for all routers and links

- System lifetime = minimum MTTF across all components
- Report bottom 10% of MTTF distribution (weakest components determine system fate)
- Compute AF vs DOR baseline

Acceleration Factor:

$$AF_{EM} = \frac{MTTF_{EM}|_{RL}}{MTTF_{EM}|_{DOR}}, \quad AF_{HCI} = \frac{MTTF_{HCI,PV}|_{RL}}{MTTF_{HCI,PV}|_{DOR}}$$

Unknown constants and I_{sub} cancel in the ratio since PV is identical across both runs. You only need E_a values (published), utilization, and temperature from simulation.

Section 3: Clotho's Routing Algorithm

The Core Idea

Clotho is a **proactive** wear-aware routing algorithm. Unlike fault-tolerant routing which reacts to failures after they occur, Clotho steers traffic away from components that are most susceptible to wear **before they fail**. It does this by continuously monitoring utilization, temperature, and PV across the network and routing packets through the healthiest paths.

Clotho-LAR (Locally Adaptive Routing)

LAR makes routing decisions using **only nearest-neighbour information** — the current router and its immediate neighbours' wear state, temperature, and utilization.

How it works:

- At each hop, the router looks at the wear state of its directly connected neighbours
- Picks the output port leading to the least-worn immediate neighbour
- No knowledge of what lies beyond the next hop

Limitation — myopia: LAR cannot see that a healthy-looking next hop leads into a heavily worn region further along the path. It may pick a direction that looks good locally but leads to a terrible path overall.

Result from paper: Only 3% average improvement in $MTTF_{EM}$ and actually a **5% reduction** in $MTTF_{HCI}$ — the myopia actively hurts HCI

because locally good decisions compound into globally bad paths.

Clotho-GAR (Globally Adaptive Routing)

GAR takes a **full network view** — every router knows the wear state of every other router and link in the network.

How global state is communicated: This is the key mechanism. GAR uses **header flit piggybacking** — as packets traverse the network, each router appends its local wear metrics (utilization, temperature) into empty space in the packet's header flit. When the packet arrives at a router, that router reads the piggybacked information and updates its routing map. Information from one router propagates to all routers over time as packets flow through the network.

How routing decisions are made: At each hop, GAR computes W_{norm} — a score for every candidate minimal path from current position to destination:

$$W_{norm} = \sum_{min_paths} [C_{EM} \cdot MTTF_{EM,norm_i} + C_{HCI,PV} \cdot MTTF_{HCI,PV,norm_i}]$$

This sums the weighted normalized MTTF of every router and link along each candidate path. The path with the highest W_{norm} is chosen — meaning the path whose **cumulative health** across all components is greatest.

The weighting function $C(MTTF)$ exponentially penalizes nearly-exhausted components:

$$C(MTTF) = \frac{\exp\left(w \cdot \frac{MTTF - MTTF_{max}}{MTTF_{max} - MTTF_{min}}\right) - \exp(-w)}{1 - \exp(-w)}, \quad w = 8$$

This ensures a single heavily worn component drastically reduces a path's score — the weakest link logic.

Route computation uses a **modified Bellman-Ford** algorithm to find the minimal-hop path with highest W_{norm}. Deadlock freedom is maintained via Duato's protocol.

Result from paper: 66% average improvement in $MTTF_{EM}$ and 8% in $MTTF_{HCI}$ — dramatically better than LAR.

GAR vs LAR — The Core Difference

	Clotho-LAR	Clotho-GAR
State visibility	Nearest neighbours only	Full network
Path scoring	Next hop only	Full path enumeration
State communication	None needed	Header flit piggybacking
Computational cost	Low	Higher (Bellman-Ford per decision)
EM improvement	3%	66%
HCI improvement	-5% (worse)	8%

The dramatic difference proves that **global visibility is essential** for effective wear-aware routing. Local information is simply insufficient to make good long-term wear decisions.

Section 4: My RL Routing Agent

How It Differs From Clotho

Your agent is **global like GAR** but fundamentally different in mechanism:

	Clotho-GAR	Your RL Agent
Global state source	Header flit piggybacking	Centrally stored in GarnetNetwork
Path scoring	Explicit W_{norm} full path enumeration	Implicit via learned Q-values
Decision mechanism	Bellman-Ford shortest path	Q-table lookup
Adaptability	Reactive to current state only	Learns patterns over time
Path scope	Full path computed upfront	Per-hop greedy, learns long-term value

Your key contribution: the Q-values **implicitly learn** the equivalent of W_{norm} through experience, without ever explicitly enumerating full paths.

State Space

At each routing decision the agent observes:

Component	Values	Bins
src_router_id	0 to N-1	16 (for 8×8 mesh)
dest_router_id	0 to N-1	16
HCI wear of horizontal candidate router	$MTTF_{HCI,norm}$	3 (GOOD≥0.67, FAIR∈[0.33,0.67), POOR<0.33)
EM wear of horizontal candidate link	$MTTF_{EM,norm}$	3
HCI wear of vertical candidate router	$MTTF_{HCI,norm}$	3
EM wear of vertical candidate link	$MTTF_{EM,norm}$	3
Congestion of horizontal candidate port	active VCs / total VCs	3 (LOW/MED/HIGH)
Congestion of vertical candidate port	active VCs / total VCs	3

Total states: $16 \times 16 \times 3^6 = 186,624$

The state captures **what lies ahead** — the current router is already committed, what matters is the wear and congestion of the candidate next hops.

Action Space

Only two actions — always minimal path preserving:

- **Action 0:** Go horizontal (East if dest_x > my_x, West otherwise)
- **Action 1:** Go vertical (North if dest_y > my_y, South otherwise)

Q-table is only consulted when **both dx≠0 and dy≠0** — when only one dimension remains, routing is forced and no decision is needed.

Reward Function

Per-hop, computed immediately at routing time:

$$R_{hop} = w_{wear} \cdot MTTF_{score,chosen} - w_{latency} \cdot \frac{cong_{chosen}}{2}$$

Where:

$$MTTF_{score,chosen} = C_{EM} \cdot MTTF_{EM,norm,chosen} + C_{HCI} \cdot MTTF_{HCI,norm,chosen}$$

- $w_{wear} = 0.85$, $w_{latency} = 0.15$ — wear dominates, matching Clotho's priority
- Congestion bin used as latency proxy since fixed hop cost doesn't differentiate actions
- All inputs available immediately at routing time — no waiting for packet delivery

Q-Learning Update

Per-hop, self-contained inside `outportComputeCustom()`:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R_{hop} + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right]$$

- **s'** computed immediately from GarnetNetwork snapshots for the next router
- **Terminal state** (next router = destination): $\gamma \cdot \max Q(s') = 0$
- $\alpha = 0.1$, $\gamma = 0.95$, $\epsilon = 0.1$ (decaying over training)
- No pending experience table needed — everything observable at routing time

Training vs Evaluation

Training phase: Agent runs with ϵ-greedy exploration — occasionally takes random actions to discover better paths. Q-table continuously updated every hop.

Evaluation/benchmarking phase: $\epsilon = 0$ — pure exploitation. Agent always takes greedy best action from learned Q-table. This is when you collect MTTF data for AF computation.

Section 5: Evaluation

Benchmarks

Use **PARSEC benchmark suite** — exactly as Clotho does, enabling direct comparison:

Benchmark	Abbreviation	Characteristics
blackscholes	bls	High EM improvement in Clotho (114%)
fluidanimate	fla	High EM improvement in Clotho (114%)

Benchmark	Abbreviation	Characteristics
x264	x264	High HCI improvement in Clotho (16%)
bodytrack	bdt	High HCI improvement in Clotho (12%)

Run each benchmark under:

1. **Baseline DOR** — standard XY dimension-order routing
2. **Clotho-GAR** — if you can replicate it, for direct comparison
3. **Your RL agent** — trained then evaluated

Plots to Make

1. EM Acceleration Factor (primary result)

Bar chart — one bar per benchmark, comparing AF_{EM} under your RL agent vs Clotho-GAR vs DOR baseline (=1.0). This is your headline result — you want bars above Clotho-GAR or at least comparable.

2. HCI Acceleration Factor (primary result)

Same format as above for $AF_{HCI,PV}$.

3. MTTF Distribution (bottom 10%)

Box plot or CDF of per-component MTTF values across all routers and links, showing the bottom 10% tail. Compare RL vs DOR — the tail shifting right means weakest components are living longer.

4. Performance Impact

Bar chart of execution cycle count normalized to DOR — shows how much performance your agent sacrifices for wear improvement. Clotho-GAR costs ~2% — you want to be in that range or better.

5. Q-value Convergence

Line plot of average Q-value magnitude over training time — shows the agent is actually learning and stabilizing. Important for thesis credibility.

6. Wear Heatmap

Spatial heatmap of per-router MTTF across the 8×8 mesh under DOR vs your RL agent. Under DOR you expect center routers to be heavily worn (they carry more traffic). Under your agent the wear should be

more spatially uniform — this is a visually compelling result that directly demonstrates the algorithm working.

7. Sensitivity Analysis — Reward Weights

Line plot showing AF_{EM} and AF_{HCI} as you vary w_{wear} vs $w_{latency}$ ratio. Mirrors Clotho's EM/HCI weight sensitivity analysis in Fig. 6. Shows robustness of your approach.

8. Training Convergence — AF vs Training Episodes

Shows how AF improves as the agent trains — demonstrates learning progression and helps justify training duration choices.

How It All Connects

```
PV (static, per router)
    → precompute I_sub at init
    → feeds MTTF_HCI_PV

Temperature (synthetic, per router, updated 10⁷ cycles)
    → feeds both MTTF_EM and MTTF_HCI_PV

Utilization (windowed snapshots, updated 10⁴ cycles)
    → U_link  → α_link  → MTTF_EM  per link
    → U_router → α_router → MTTF_HCI per router

MTTF_EM + MTTF_HCI (normalized)
    → wear bins → part of RL state s
    → MTTF_score → part of reward R_hop
    → accumulated over simulation → AF computation at end

RL agent
    → observes s (wear bins + congestion + src/dest)
    → selects action (horizontal or vertical)
    → gets reward R_hop immediately
    → observes s' from GarnetNetwork snapshots
    → updates Q(s,a) in place
    → over time learns to route through high-MTTF paths
    → implicitly approximates Clotho-GAR's W_norm without path enumeration

Evaluation
    → run PARSEC under DOR and RL agent
    → compute AF_EM and AF_HCI
    → plot wear heatmaps, performance impact, convergence
    → compare against Clotho-GAR results from paper
```

March 12, 2026

Implementation of RL-Based Wear-Aware Routing in gem5

Today, I completed a full implementation of a reinforcement learning based wear-aware routing algorithm in the gem5 Garnet2.0 NoC simulator. The goal of this implementation was to replace the default routing decision logic with a Q-learning based approach that balances reliability (through aging models) and congestion.

The implementation required modifying multiple components of the Garnet2.0 infrastructure. The main changes were made in `GarnetNetwork`, `RoutingUnit`, `Router`, and related classes. I introduced a new data structure to store per-router process variation parameters, including threshold voltage variation, effective channel length variation, and a precomputed subthreshold leakage current. These values are initialized once using both random and spatially correlated components through a Cholesky decomposition-based approach. This ensures that process variation is realistic and consistent across runs.

I implemented temperature modeling and utilization tracking as part of the runtime system state. Router utilization is derived from crossbar activity, while link utilization is obtained from link traversal statistics. These values are updated periodically, and a spatially correlated temperature map is also regenerated at large time intervals. This allows the aging models to reflect dynamic operating conditions.

Two aging models were integrated into the routing decision process. Electromigration (EM) is modeled per link and depends on link utilization and temperature, with a quadratic dependence on switching activity. Hot Carrier Injection (HCI) is modeled per router and depends on utilization, temperature, and the precomputed process variation parameter. Both models produce proportional MTTF values that are used during routing decisions.

In the routing logic, I replaced the placeholder function `outportComputeCustom()` with a full Q-learning implementation. At each routing step, the router evaluates possible output directions based on normalized MTTF values and congestion levels. These are combined using an exponential weighting function inspired by prior work to produce a reliability score. The state representation includes source and destination nodes, wear levels for both EM and HCI in

candidate directions, and congestion levels. This results in a discrete state space of size 186,624.

An epsilon-greedy policy is used for action selection, with a configurable warmup period during which the system falls back to deterministic XY routing. After the warmup phase, the Q-table is updated online at every routing decision using a reward function that prioritizes higher MTTF while penalizing congestion.

I also implemented mechanisms for saving and loading the Q-table to allow training and evaluation runs to be separated. At the end of each simulation, the system writes out per-router and per-link MTTF values to a file. These outputs are used later to compute acceleration factors by comparing against a baseline deterministic routing run.

March 19, 2026

Issues with Building gem5 on Windows

Today, I spent time trying to set up and build the gem5 simulator on my Windows machine. I went through several online resources and followed multiple tutorials to get the build environment working properly. Despite this, I kept running into different errors during the build process. Each time I resolved one issue, another would appear, which prevented the build from completing successfully.

After several attempts, it became clear that continuing with the Windows setup was not an efficient use of time. The build process for gem5 does not seem to be well-supported on Windows, and most of the documentation and community support assume a Linux-based environment.

Based on this, I decided to stop trying to build gem5 on Windows and instead switch to using my Mac for development. However, this introduces some practical concerns. My Mac currently has limited available storage, and building gem5 is resource-intensive, which may result in slower performance during both compilation and simulation.

Moving forward, I will attempt to set up gem5 on the Mac and monitor whether the storage and performance limitations become a significant issue.

March 26, 2026

Building and Testing gem5 with RL Routing

Today, I successfully built gem5 on my Mac. The process was straightforward compared to the issues I faced on Windows. I used the command:

```

scons build//gem5.opt

```

The ISA can be set to options like `ALL`, `NULL`, `x86`, etc., as listed on the gem5 website. For initial testing, I used the `NULL` ISA since I only wanted to verify that the simulator builds and runs correctly with default settings.

After building, I ran a basic test using deterministic dimension-order routing (DOR), which corresponds to XY routing. The simulation completed successfully and produced output files. I was able to observe the MTTF results for both routers and links. These MTTF values are proportional rather than absolute, so they do not have physical units, but they are still useful for comparison. This confirmed that the baseline setup is working as expected.

I then proceeded to test the reinforcement learning based routing algorithm. This required running the simulator with the `--algorithm=2` flag. During this step, I encountered an issue related to how I had defined the constructor and destructor in `GarnetNetwork`. The destructor was effectively empty, which meant that the Q-table and MTTF outputs were not being properly generated at the end of the simulation. After correcting this, I reran the simulation.

The next issue took significantly longer to debug. There was a typo in the reward function, which occasionally caused a division by zero during computation. This error was not immediately obvious and required careful inspection of the code to identify. After fixing the typo and ensuring the reward calculation was stable, I ran the simulation again.

This time, the RL routing executed correctly. I was able to observe MTTF values for both routers and links under the RL policy. When compared to the DOR baseline, the results showed improvement. The ratio of (\text{MTTF}*{RL} / \text{MTTF}*{DOR}) was approximately:

- 1.5 for EM
- 1.05 for HCI

These results are promising and indicate that the RL-based routing is successfully improving reliability, especially for EM, which is more sensitive to traffic distribution.

March 30, 2026

Issues with RL Training and Deadlock Handling

Today, I attempted to further train the RL agent to improve the MTTF results. The idea was to allow more exploration so that the agent could discover better routing policies over time. However, this led to a major issue. During exploratory training, the routing decisions occasionally resulted in deadlock situations very quickly. When this happens, the simulator panics and terminates the run.

This created a bigger problem because the RL agent would sometimes learn poor Q-values during these failed runs. Since the training process was interrupted, I could not properly retrain or recover from these bad policies. As a result, simply increasing exploration was not a viable approach.

To address this, I revisited the LARE paper and looked at how they handle deadlock. They use an escape virtual channel (VC), which follows deterministic XY routing. The purpose of this escape VC is to guarantee forward progress by routing packets away from congested regions using a deadlock-free path.

Based on this idea, I updated my RL routing implementation to include an escape VC mechanism similar to the one described in the LARE paper. This ensures that even if the RL policy makes poor decisions, there is always a fallback path that avoids deadlock.

At the same time, I also modified the RL implementation to follow the structure used in LARE. Instead of maintaining a large Q-table for all possible states, I moved toward a more compact representation using feature-based approximation. This significantly reduces memory overhead and makes the approach more scalable.

These changes stabilize the training process and make the RL routing more practical to use within the simulator.

Background on LARE

Instead of storing a global Q-table, LARE uses a linear function to approximate Q-values. Each router stores a small coefficient vector

per output port. The Q-value for a given state and output port is computed as a dot product:

$$Q_{x,out}(s) = \sum_j \theta_j \cdot f_j$$

The coefficient update rule (instead of Q-table update) is:

$$\theta_j \leftarrow \theta_j + \alpha(r + \gamma Q(s') - Q(s)) \cdot f_j$$

This eliminates the Q-table entirely — each router stores only a few floats per output port instead of a large state-action table.

New Design

1. State Feature Vector

Replace the 8-component binned state index with a 9-feature float vector per output port candidate. Features are normalized to [0,1] by dividing by their range:

Feature	Description	Range	Normalization
f_0	Constant term	1	1
f_1	Destination router ID	0–63	divide by 63
f_2	Current router ID	0–63	divide by 63
f_3	Hops traversed so far	0–14	divide by 14
f_4	Remaining hops to destination	0–14	divide by 14
f_5	Normalized EM MTTF of candidate link	0–1	already normalized
f_6	Normalized HCI MTTF of candidate router	0–1	already normalized
f_7	Congestion of candidate output port	0–1	divide by 2 (bin/2)
f_8	Input port direction encoded	0–1	divide by 4

For an 8×8 mesh, maximum Manhattan distance is 14 hops (7+7). `hops_traversed` is available from `route.hops_traversed`. Remaining hops = Manhattan distance from current router to destination = `|dx| + |dy|`.

2. Coefficient Storage (replaces Q-table)

Each router stores one coefficient vector per output port (East, West, North, South — 4 ports × 9 features = 36 doubles per router). For 64 routers: 64 × 36 × 8 bytes = ~18 KB total. Negligible.

Store in GarnetNetwork as:

```C++
// [router_id][outport_idx][feature_idx]
std::vector<std::vector<std::vector<double>>> m_lare_theta;
// Size: num_routers × num_outports × NUM_FEATURES
// Initialized to 1.0 as per the paper
static const int LARE_NUM_FEATURES = 9;
```

3. Q-value Computation

For a given router R, output port `out`, and state feature vector `s`:

```C++
double computeQLARE(int router_id, int outport_idx,
                    const std::vector<double>& features) const;
// Returns: dot product of theta[router_id][outport_idx] with features
```

4. Action Selection (epsilon-greedy, minimal path only)

Same as before — only horizontal and vertical minimal path directions are considered. The agent computes Q-value for each candidate direction using `computeQLARE()` and picks the higher one (or explores randomly with probability epsilon).

5. Reward Function

Combine wear-awareness with congestion/latency signal:

$$R = w_{wear} \cdot MTTF_{score,chosen} - w_{latency} \cdot cong_{chosen} + w_{balance} \cdot (1 - cong_{chosen})$$

Where:

- $MTTF_{score,chosen} = C_{EM} \cdot MTTF_{EM,norm} + C_{HCI} \cdot MTTF_{HCI,norm}$ of chosen direction (same as before)
- $cong_{chosen}$ = congestion bin of chosen port normalized to [0,1] by dividing by 2
- $w_{wear} = 0.7$, $w_{latency} = 0.15$, $w_{balance} = 0.15$

6. Coefficient Update Rule

```C++
```

```
// After taking action, computing reward, and observing s':
double q_s  = computeQLARE(my_id, chosen_outport, features_s);
double q_sp = computeQLARE(next_id, best_outport_at_next, features_sp);
double td_error = R + gamma * q_sp - q_s;

for (int j = 0; j < LARE_NUM_FEATURES; j++) {
    m_lare_theta[my_id][chosen_outport][j] +=
        alpha * td_error * features_s[j];
}
```

Terminal state: `q_sp = 0`.

7. Deadlock Freedom — Escape VC Mechanism

Based on Duato's theory and the LARE paper. Designate **VC 0** in each vnet as the escape VC. The escape VC always uses XY routing regardless of the Q-values. The RL agent only makes decisions on VCs 1 and above.

In `outportComputeCustom()`:

```C++
// Check if this packet is on the escape VC (VC 0 in its vnet)
int vnet = route.vnet;
int vc   = route.vc_choice;  // or however current VC is accessible
int vcs_per_vnet = net->getVCsPerVnet();
int local_vc = vc % vcs_per_vnet;  // VC index within its vnet

if (local_vc == 0) {
    // Escape VC — use XY routing for deadlock freedom
    return outportComputeXY(route, inport, inport_dirn);
}
// VC 1+ — use LARE routing
```

Please check the actual RouteInfo and flit structures to confirm how to access the current VC assignment for a packet at routing decision time. Look at how `vc_choice` or equivalent is set in the flit/route when `outportComputeCustom()` is called.

8. Coefficient Persistence (replaces Q-table save/load)

Same binary file mechanism but saving `m_lare_theta` instead of `m_q_table`:

```C++
```

```
void saveLARETheta() const;
void loadLARETheta();
```

File size: 64 × 4 × 9 × 8 bytes = ~18 KB.

9. MTTF Report

Unchanged — still write per-component proportional MTTF values at simulation end for AF computation.

What Needs To Change vs Current Implementation

Component	Current	New
State encoding	8-bin index → flat Q-table index	9-feature float vector
Q-table	`m_q_table` 186,624 × 2 global array	`m_lare_theta` 64 × 4 × 9 per-router coefficients
Q-value lookup	`qtable[s][action]`	dot product `θ · f`
Q-update	Table entry update	Coefficient vector update
Deadlock	Not guaranteed	Escape VC 0 → XY routing
Memory	2.9 MB	~18 KB
Mesh scalability	Hardcoded 16 routers	Works for any mesh size
Save/load	`saveQTable/loadQTable`	`saveLARETheta/loadLARETheta`

April 1, 2026

Correct Build Configuration and Improved RL Results

Today, I realized that I had been building gem5 incorrectly. The correct setup for my use case requires using the **Garnet Standalone** protocol, which is specified through a configuration change rather than the standard build process I had been using earlier.

After updating the configuration to use Garnet Standalone, I rebuilt gem5 and reran the simulations with the updated RL routing agent.

The results improved significantly compared to my previous runs. The acceleration factors are now approximately:

- AF_HCI ≈ 2.1
- AF_EM ≈ 3.1

These values are much higher than what I observed earlier, where the improvements were relatively small. They also outperform the results reported in the Clotho paper, where the acceleration factors were approximately:

- AF_HCI ≈ 1.03
- AF_EM ≈ 2.20

This confirms that the corrected build configuration was critical, and that the RL-based routing approach is now producing strong reliability improvements for both EM and HCI.